
τ -bench: 실제 domain에서의 Tool-Agent-User 상호작용 benchmark

Shunyu Yao* Noah Shinn Pedram Razavi Karthik Narasimhan

Sierra

Abstract

기존 benchmark는 language agent가 인간 user와 상호작용하는 방식이나 domain-specific 규칙을 따르는 능력을 평가하지 않는데, 이 둘은 실제 애플리케이션에 agent를 배치하는 데 모두 핵심적이다. 우리는 domain-specific API tool과 policy guideline이 주어진 language agent와 user(language model이 시뮬레이션함) 사이의 동적인 대화를 모사하는 benchmark인 τ -bench를 제안한다. 우리는 대화 종료 시점의 database state를 주석된 목표 state와 비교하는 효율적이고 faithful한 evaluation 절차를 사용한다. 또한 여러 trial에 걸친 agent 행동의 신뢰성을 평가하기 위한 새로운 metric(pass^k)을 제안한다. 실험 결과, state-of-the-art function calling agent(gpt-4o 등)조차 $< 50\%$ 의 task만 성공하며, 일관성도 상당히 낮다(retail에서 $\text{pass}^8 < 25\%$). 우리의 결과는 agent가 일관되게 행동하고 규칙을 신뢰성 있게 따르는 능력을 개선할 수 있는 방법이 필요함을 보여준다.

1 서론

language agent가 여러 산업 전반에서 새로운 수준의 자동화를 가능하게 할 잠재력에 대한 기대가 커지고 있다 [20, 24, 18, 1]. 그러나 이를 실제 시스템에 배치하려면 몇 가지 핵심 desiderata를 충족해야 한다. agent는 (1) 긴 horizon에 걸쳐 인간과 programmatic API 모두와 자연스럽게 상호작용하며 정보를 점진적으로 수집하고 intent를 해결해야 하고, (2) task 또는 domain에 특화된 복잡한 policy와 규칙을 정확히 준수해야 하며, (3) 수백만 건의 상호작용 규모에서도 일관성과 신뢰성을 유지해야 한다. 예를 들어 항공 예약 agent의 경우를 생각해 보자(그림 1). user가 자신의 항공편 reservation을 다른 목적지 공항으로 변경하고 싶어 할 때, agent는 user와 상호작용해 필요한 정보를 모으고, 제공된 guideline으로 항공사 policy를 확인하며, 복잡한 reservation API를 사용해 새 항공편을 찾고 가능하다면 user를 다시 예약해야 한다. 또한 agent는 동일한 요청을 하는 서로 다른 유형의 user에게도 일관되게 행동해야 하며, 최종 결과에 영향을 주지 않아야 하는 대화 흐름의 작은 변화에도 robust해야 한다.

agent evaluation에서 현실적인 인간 상호작용과 규칙 준수를 모델링하는 일은, 실제 환경에서 신뢰할 수 있는 agent를 개발하고 배치하는 데 필수적이며, long-context reasoning과 planning 같은 과제를 체계적으로 다루는 데도 중요하다. 기존의 language agent benchmark [27, 29, 12, 14, 16]는 흔히 단순화된 instruction-following 설정을 사용한다. 이 설정에서는 agent가 모든 정보를

*인턴십 기간 중 수행한 작업. Code와 data: <https://github.com/sierra-research/tau-bench>.

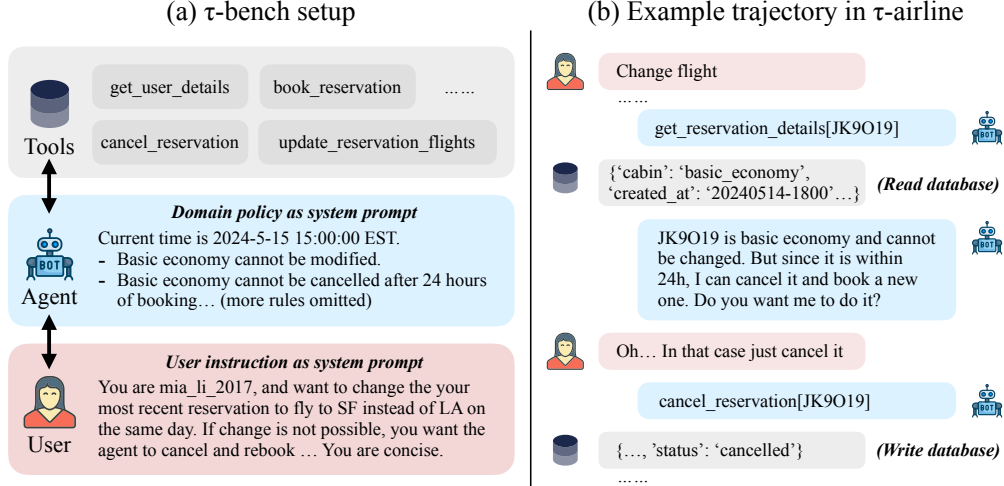


Figure 1: (a) τ -bench에서 agent는 task를 완료하기 위해 database API tool 및 **LM-simulated user**와 상호작용한다. 이 benchmark는 agent가 여러 번의 상호작용을 통해 user로부터/에게 필요한 정보를 모두 수집하고 전달하는 능력, 그리고 domain-specific policy 문서에 제시된 **guideline**을 준수하면서 즉석에서 복잡한 문제를 해결하는 능력을 평가한다. (b) τ -airline의 trajectory 예시. 여기서 agent는 domain policy를 따라 user 요청(기본 이코노미 항공권 변경)을 거절하고 새로운 해결책(취소 후 재예약)을 제안해야 한다. 이는 복잡한 database, 규칙, user intent에 대한 long-context zero-shot reasoning을 agent에게 요구한다.

처음부터 받은 상태에서 환경(web, code terminal, 또는 API)과 자율적으로 상호작용하며, human-in-the-loop 상호작용도 없고 domain-specific guideline을 참고할 필요도 없다.

이 연구에서 우리는 τ -bench(Tool-Agent-User Interaction Benchmark의 약자)를 소개한다. 이는 domain-specific policy를 일관되게 따르면서 (시뮬레이션된) 인간 user 및 programmatic API와 상호작용하는 agent의 능력을 측정하기 위한 benchmark다. τ -bench는 (1) 현실적인 database와 API, (2) domain-specific policy 문서, (3) 다양한 user 시나리오를 위한 instruction과 그에 대응하는 ground truth annotation으로 이루어진 modular framework 위에 구축된다. 첫 번째 시연으로, 우리는 customer service 영역에 초점을 맞춰 agent가 다양한 요청을 가진 시뮬레이션 user를 도와야 하는 두 개의 서로 다른 domain(τ -retail과 τ -airline)을 만든다. 우리는 data 생성과 현실적인 인간 user simulation을 위해 language model(LM)의 생성 능력 [15]을 활용하고, 이를 수작업 annotation 및 verification과 결합한다.

τ -bench는 세 단계로 구축했다. 수작업 schema 및 API 설계, LM 보조 data entry 생성, 그리고 user simulator를 위한 수작업 시나리오 생성 및 verification이 그것이다. 우리의 evaluation scheme은 각 episode가 끝났을 때의 database state를 ground truth 기대 state와 비교한다. 이렇게 하면 agent의 의사결정을 객관적으로 측정할 수 있고, 동시에 대화 자체의 stochastic variation도 허용할 수 있다. user가 같은 요청을 서로 다른 방식으로 표현하더라도 database의 최종 state는 같을 수 있기 때문이다. 또한 우리는 k 개의 i.i.d. trial 전반에서 agent의 일관성과 robust함을 측정하는 metric인 pass^k 를 도입한다.

우리의 실험은 function calling이나 ReAct 같은 단순한 LM 구성 요소로 만든 agent의 성능이 낮음을 보여주며, 더 정교한 agent architecture가 필요하다는 점을 부각한다. 예를 들어, gpt-4o 같은 state-of-the-art LM조차 function calling을 사용할 때 낮은 task 성공률(pass^1)을 보인다(τ -retail에서 $\sim 61\%$, τ -airline에서 $\sim 35\%$). k 가 증가할수록 task를 일관되게 해결할 확률은 빠르게 떨어져, 같은 모델의 τ -retail pass^8 은 $\sim 25\%$ 까지 낮아진다. 이는 인간-agent 상호작용에서 흔한 stochasticity와 부분 정보에 이런 agent가 얼마나 취약한지를 보여준다. 실패 사례를 분석해 보면, 현재 agent는 database에 대한 복잡한 reasoning, ad-hoc policy의 이해와 준수, compound(둘 이상) 요청 처리에 어려움을 겪는다. 우리는 τ -bench가 인간 상호작용이 포함된 실제 디지털 task를 위해 더 일관되고 더 유능한 agent를 평가하고 개발하는 데 기여하길 바란다.

2 관련 연구

기존의 agent 및 task-oriented dialogue system benchmark 대부분은 conversational 능력이나 tool-use 능력 중 하나만 평가하는 데 초점을 둔다. τ -bench는 이 둘을 보다 현실적인 설정 아래 통합하고, 동시에 agent가 domain-specific policy를 얼마나 일관되게 따를 수 있는지도 시험하는 것을 목표로 한다.

language agent 및 tool use를 위한 benchmark. LM으로 구동되는 agent를 평가하기 위한 여러 benchmark가 개발되어 왔다 [27, 29, 12, 14, 16]. 최근의 노력은 특히 LM의 tool use 능력, 즉 API 안의 함수 집합에서 올바른 function call을 생성하는 능력을 평가하는 데 집중해 왔다. Berkeley Function Calling Leaderboard(BFCL) [23], ToolBench [22], MetaTool [11] 같은 프로젝트는 여러 프로그래밍 언어에서 tool use/function calling을 시험하고 function call의 정확도를 평가하는 다양한 방법을 제안한다. ToolEmu [16]는 language model 자체를 사용해 tool 실행을 에뮬레이션하며, LM agent가 tool을 잘못 사용할 때 드러나는 잠재적 안전성 위험을 노출하는 데 초점을 둔다. 그러나 이 모든 연구는 단일 단계의 user 상호작용만 포함하며, 이때 agent와 상호작용하는 인간은 필요한 모든 정보를 담은 초기 instruction 하나를 제공한다. 반면 우리의 benchmark는 agent가 정보와 authorization을 수집하기 위해 인간 user와 상호작용해야 하는 보다 현실적인 설정에 초점을 둔다.

Task-oriented dialogue. Task-oriented dialogue는 NLP에서 오랫동안 이어진 도전 과제로, 여러 해에 걸쳐 domain-specific offline dataset이나 user simulator를 구축하려는 시도가 있었다. 전자의 benchmark는 정적이며, 사전에 수집된 conversation trajectory에서만 conversational agent를 시험한다 [4, 3, 2]. 후자는 rule-based user simulator에 의존하거나 symbolic specification에 기반하거나 [17, 8], crowdsourcing 플랫폼을 통해 실제 인간으로 테스트를 수행한다 [9]. 아주 최근의 일부 연구는 dialogue system을 학습시키기 위해 LM을 response rater로 사용하는 방식을 탐구하거나 [10], user 시뮬레이션 능력을 평가한다 [7]. τ -bench는 state-of-the-art LM의 강력한 텍스트 생성 능력을 활용해 *textual* 시나리오 설명만으로 현실적인 user 발화와 long-context 대화를 시뮬레이션하며, 목표는 agent를 평가하는 데 있다. LM의 stochastic sampling은 정확히 같은 시나리오를 다시 실행할 때도 다양하지만 faithful한 대화 variation을 허용하는데, 이는 § 5.1에서 보이듯 agent의 일관성을 시험하는 데 매우 유용하다.

LM을 활용한 user simulation. 우리의 연구는 LM을 인간 캐릭터 simulator로 사용하는 최근의 노력과도 관련이 있다. 여기에는 텍스트 어드벤처 게임의 non-player character(NPC)를 시뮬레이션하는 연구 [13], 인간 사회와 유사한 환경의 multiple agent [15] 또는 특정 협업 task [21]를 다루는 연구, 그리고 온라인 쇼핑 [6]이나 웹 검색 [28] 같은 task에서 human-in-the-loop 상호작용을 가능하게 하는 연구가 포함된다. 그러나 이러한 선행 연구는 이런 simulator를 agent의 신뢰성을 benchmark하는 데 사용하지 않았고, 대신 LM이 현실적인 simulation을 가능하게 한다는 점을 보여주는 데 초점을 맞추었다. 우리의 연구는 이런 현실적인 user simulation을 사용해, 실제 환경에서 수백만 건의 인간 상호작용을 처리하는 시스템에 AI agent를 배치할 때 필요한 신뢰성과 robustness를 정확히 평가한다.

3 τ -bench: Tool-Agent-User 상호작용을 위한 benchmark

τ -bench의 각 개별 task는 상태 공간 $\mathcal{S}$, 행동 공간 $\mathcal{A}$, 관측 공간 $\mathcal{O}$, 전이 함수 $\mathcal{T} : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S} \times \mathcal{O}$, 보상 함수 $\mathcal{R} : \mathcal{S} \rightarrow [0, 1]$, instruction 공간 $\mathcal{U}$ 를 갖는 부분 관측 마르코프 결정 과정(POMDP) $(\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, \mathcal{R}, \mathcal{U})$ 으로 정식화할 수 있다. agent는 task를 완료하기 위해 (1) API tool을 통해 database(db)와, (2) (시뮬레이션된) user($user$)와 모두 상호작용한다. 즉, $\mathcal{S} = \mathcal{S}_{db} \otimes \mathcal{S}_{user}$, $\mathcal{A} = \mathcal{A}_{db} \cup \mathcal{A}_{user}$, $\mathcal{O} = \mathcal{O}_{db} \cup \mathcal{O}_{user}$ 이다. 또한 agent에는 반드시 준수해야 하는 규칙을 담은 domain-specific policy 문서가 제공되는데, 이는 domain의 world model을 부분적으로 설명하는 것으로 볼 수 있다. 아래에서는 각 구성 요소를 더 자세히 설명한다.



Figure 2: τ -bench는 여러 구성 요소를 갖는 modular한 방식으로 구축된다: (a) JSON database, (b) Python API tool, (c) Markdown domain policy, (d) JSON task instance. agent는 API tool과 domain policy만 직접 접근할 수 있고, API tool을 통해서만 database에 간접 접근할 수 있다. Task annotation은 agent에게 보이지 않으며 user simulation과 evaluation에만 사용된다.

Database와 API. 각 τ -bench domain에는 여러 database와 그에 대응하는 API가 있다. database의 내용은 상태 s_{db} 를 이룬다(그림 2a). 이 상태는 agent와 user에게 숨겨져 있으며, 보통 `tool_name(**kwargs)` 형태인 API action a_{db} 를 사용해야만 읽거나 쓸 수 있다. database에서 action이 실행되면 전이 $\mathcal{T}_{db} : (s_{db}, a_{db}) \mapsto (s'_{db}, o_{db})$ 는 결정적이며 Python 함수로 구현된다(그림 2b).

Domain policy. 각 domain에는 agent가 상호작용 중 따라야 할 domain database, task 절차, 제약을 설명하는 policy가 있다(그림 2c). 일부 제약은 API 내부의 check로 구현된다. 예를 들어 user profile에 없는 payment ID를 사용하면 $o_{db} = \text{"Error: payment not found"}$ 가 된다. 반면 일부 제약은 그렇지 않다. 예를 들어 airline policy는 membership status와 cabin class에 따라 서로 다른 수하물 허용량을 규정하지만, agent는 실제 환경의 agent에게 주어지는 자유와 비슷하게 `book_reservation` API에 결제 대상 수하물 개수를 직접 채워 넣어야 한다.

User simulation. 우리는 agent와 상호작용하는 인간 user를 시뮬레이션하기 위해 language model(gpt-4-0613)을 사용한다. user 상태 s_{user} 는 task instruction이 들어 있는 초기 system prompt(그림 2d)와, 현재까지의 user-agent 전체 대화 이력으로 구성된다. user는 agent와 API tool 사이의 상호작용 이력을 볼 수 없다. agent는 어떤 자연어 메시지로도 user와 상호작용할 수 있으며, 예를 들어 a_{user} 는 "Your reservation has been updated, is there anything else I can help with?"가 될 수 있다. 전이 $\mathcal{T}_{user} : (s_{user}, a_{user}) \mapsto (s'_{user}, o_{user})$ 는 stochastic하며, agent의 메시지를 채팅 이력에 붙인 뒤 LM으로부터 새로운 user 메시지를 샘플링한다. 예를 들어 그 결과 o_{user} 는 "Yes, I also want to cancel another flight."가 될 수 있다. user가 $o_{user} = \text{"###STOP###"}$ 를 내보내면 episode가 종료되고 agent를 평가한다.

Task instance. 그림 2d에 보이듯이, 각 τ -bench task instance는 두 부분으로 구성된다. 하나는 user simulation을 위한 instruction(agent에게는 숨겨짐)이고, 다른 하나는 ground truth database write action의 annotation(그리고 선택적으로는 user 질문에 대한 ground truth output)이다. instruction은 user의 신원, intent, 선호를 설정해 domain policy 아래에서 가능한 outcome이 정확히 하나가 되도록 만든다. 각 task episode는 시뮬레이션된 user가 요청을 시작하는 것으로 이루어지며, agent는 제공된 policy를 참고하면서 언제든지 tool을 호출할 수 있는 대화형 방식으로 이를 처리한다. episode가 끝나면 database state와 agent-to-user 메시지를 사용해 reward를 계산한다.

Reward. task episode의 reward $r = r_{\text{action}} \times r_{\text{output}} \in \{0, 1\}$ 는 (1) 최종 database가 유일한 ground truth outcome database와 동일한지(r_{action}), (2) user에 대한 agent의 응답이 필요한 모든 정보를 포함하는지(r_{output})에 기반한다. 따라서 그림 2d의 task에서는 agent-user 대화가 다양하게 전개될 수 있고 agent가 여러 (read) action을 호출할 수 있지만, 유일한 database write action이 `return_delivered_order_items(order_id="#W2890441", item_ids=["2366567022"], payment_method_id="credit_card_1061405")`이며 user 응답에 "54.04", "41.64"가 부분 문자열로 포함될 때 agent는 성공한 것이다. $r = 1$ 이 성공한 episode의 필요조건일 수는 있지만 충분조건은 아닐 수 있다는 점에 유의하자. 예를 들어 agent가 명시적인 user 확인 없이 return을 실행했다면 이는 policy 위반이다. 그럼에도 우리가 제안하는 rule-based reward는 계산이 빠르고 faithful하며, § 5에서 보듯이 현재 모델과 방법에 이미 상당한 도전을 제기한다.

Pass[^]k metric. 좋은 verification 기법(unit test 등)이 있는 code generation 같은 task의 경우, 커뮤니티는 k 개의 i.i.d. task trial 중 적어도 하나가 성공할 확률로서 `pass@k`(pass at k) metric을 정의해 왔다. 이는 inference-time compute를 늘릴수록 agent가 해법을 발견할 가능성이 높아지는 경향을 포착한다 [5]. customer service처럼 신뢰성과 일관성이 필요한 실제 agent task를 위해 우리는 새로운 metric인 `pass^k`(pass hat k)를 제안한다. 이는 k 개의 i.i.d. task trial이 모두 성공할 확률을 task 평균으로 정의한다. 따라서 어떤 task를 n 회 실행했고 그중 c 개의 trial이 성공했다면($r = 1$), `pass^k`와 `pass@k`의 불편 추정치는 다음과 같다.

$$\text{pass}^k = \mathbb{E}_{\text{task}} \left[\frac{\binom{c}{k}}{\binom{n}{k}} \right], \quad \text{pass}@k = 1 - \mathbb{E}_{\text{task}} \left[\frac{\binom{n-c}{k}}{\binom{n}{k}} \right].$$

우리의 경우 같은 task에서는 user prompt와 database transition은 동일하고, user 및 agent 메시지의 LM sampling만이 충분한 stochasticity를 만들어낸다. 따라서 `pass^k`는 동일한 근본 의미를 가지는 대화 variation을 처리하면서도 domain policy와 규칙을 준수하는 agent의 신뢰성을 포착할 수 있다. 기본적으로 우리는 task 전반의 평균 reward, 즉 `pass^1=pass@1= $\mathbb{E}[r] = \mathbb{E}[c/n]$` 를 agent 비교의 주요 metric으로 보고한다.

4 Benchmark 구축

τ -bench는 여러 domain이 공유하는 domain-agnostic environment 및 user simulation class를 정의하고, database JSON 파일, database API Python code와 documentation, domain policy 텍스트, task instance 형태의 domain-specific data를 제공한다. 각 domain은 LM과 code 실행, 그리고 인간의 labeling 및 checking을 혼합한 3단계 접근법으로 만들어진다.

Stage I: database schema, API, policy의 수작업 설계. 우리는 실제 대응물에서 영감을 얻고 이를 단순화하면서, 가능한 한 가장 단순한 database schema, API, policy를 공동 설계하는 것으로 시작한다. 단순성은 여러 구성 요소의 논리적 일관성과 API 및 task annotation의 용이성에 중요하다. 그럼에도 최소한으로 현실적인 domain을 만들려면 적어도 수십 개의 schema, API, 규칙이 필요하며, 이는 기존 agent에게 충분히 어려운 과제로 드러난다. 자세한 내용은 § B.1을 보라.

	τ -retail	τ -airline
Database	사용자 500명, 제품 50개, order 1,000개	사용자 500명, 항공편 300개, reservation 2,000개
API tool	write 7개, non-write 8개	write 6개, non-write 7개
Task	115	50

Table 1: τ -retail과 τ -airline의 핵심 통계.

Stage II: LM을 활용한 자동 data 생성. data schema가 준비되면 예시 entry를 하나 만들고, gpt-4를 사용해 확장 가능한 entry를 샘플링하는 체계적인 code snippet을 생성한 뒤, code의 사소한 bug를 수작업으로 다듬는다. 예시 snippet과 자세한 내용은 § B.2를 보라.

Stage III: agent 실행을 통한 수작업 task annotation 및 검증. 여기서 핵심 과제는 user instruction이 유일한 database outcome으로 이어지도록 보장하는 것이다. 예를 들어 선택호 payment method가 명시되지 않으면 user가 trial마다 다르게 답할 수 있고, 그 결과 최종 database가 달라질 수 있다. 그래서 우리는 초기 user instruction을 작성하고, gpt-4-turbo function calling agent로 trial을 실행한 뒤, trajectory를 살펴보고 user instruction을 다듬고, 모호함이 남지 않았다고 확신할 때까지 이를 반복한다(§ A의 그림 7 참조. 여기서 우리는 각 τ -retail task를 gpt-4-turbo로 > 40회 실행하고 성공률이 0이거나 낮은 task를 모두 점검한다). ground truth annotation을 위해 agent action과 output을 복사해 수정할 수 있는데, 이는 처음부터 annotation하는 것보다 쉽다.

실제로는 data나 task를 만드는 과정에서 database schema나 policy의 사소한 세부 사항을 업데이트할 수 있지만, 세 단계는 대체로 선형적이며 구축된 data는 modular 구조로 정리된다.

4.1 Domain

위 절차를 사용해 우리는 τ -retail과 τ -airline이라는 두 domain을 modular하게 구축한다. 이 두 domain을 선택한 이유는 data(예: 제품, 가격, 항공편)를 합성하기 비교적 쉽고, 상식에 기반한 policy(예: 제품 return, 수하물 허용량)를 만들기 쉬우며, 다양한 task를 허용하고, 실제 애플리케이션과도 가깝기 때문이다. 미래에 더 강력한 agent를 다루게 되면, 더 복잡한 data와 규칙을 가진 고급 domain(예: 의료, 세금, 법률)도 연구할 수 있다. 아래에서는 두 domain의 policy를 간략히 설명한다(domain의 전체 세부사항은 § B.1).

τ -retail. 이 domain에서 agent는 user가 pending order를 취소하거나 수정하고, delivered order를 return 또는 exchange하며, user 주소를 변경하거나 정보를 제공하는 일을 돕는다. 각 제품(예: 그림 2a의 “Water Bottle”)은 고유 ID를 가진 여러 item option(예: 1000ml, stainless steel, blue)을 가진다. 각 pending order는 한 번만 취소 또는 수정할 수 있고, 각 delivered order는 한 번만 return 또는 exchange할 수 있다. item은 다른 제품 유형으로 수정되거나 exchange될 수 없다. 이러한 제약은 task와 API 설계를 단순화하는 동시에, agent가 domain-specific 규칙을 따르고 action을 취하기 전에 user에게 필요한 모든 정보를 알리고 수집하도록 만든다.

τ -airline. 여기서 agent는 user가 항공 reservation을 예약, 변경, 취소하거나 refund를 받는 일을 도와야 한다. 우리는 미국 20개 도시 사이의 항공편 300개를 현실적인 소요 시간과 가격으로 구성하고, 직항 또는 1회 경유 항공편을 조회하는 API tool을 만든다. 이 domain의 policy는 τ -retail보다 더 복잡하며, payment method 조합, 위탁 수하물 허용량, 항공편 변경 및 취소 등에 관한 ad-hoc 제약을 포함한다. 이러한 제약은 membership tier와 cabin class에 따라 달라질 수도 있어, agent에게 까다로운 multi-hop reasoning 문제를 만든다.

4.2 핵심 특성

현실적인 dialogue와 tool use. 이전의 task-oriented dialogue benchmark와 비교하면, τ -bench는 LM의 발전 덕분에 더 복잡한 database와 더 현실적인 user simulation을 갖는다. 일부

trajectory는 § C.2와 § D.2에서 볼 수 있다. 특히 user instruction이 synthetic하더라도, LM을 통해 생성된 user 발화는 open-ended하며 자연스럽다.

Open-ended하고 다양한 task. 각 τ -bench domain의 data schema, API, 규칙은 실제 domain 보다 단순화되어 있지만, 매우 다양하고 open-ended하며 때로는 창의적인 task를 만들기에 충분히 풍부하다(§ A, § C.2, § D.2 참조). 중요한 점은, 우리는 양보다 질을 택한다는 것이다. § 5에서 보이듯, 고품질 task의 작은 집합을 여러 번 trial로 실행하면(pass@k metric 사용) 서로 다른 모델, 방법, 연구 과제에 대한 풍부한 통찰을 신뢰성 있게 드러낼 수 있다.

Faithful한 rule-based evaluation. 실제 agent는 같은 task에서도 trajectory가 매우 다양할 수 있고 성공 기준도 다면적이기 때문에 평가가 어렵다. 그 결과, task 해결 여부는 최종 user가 판단하고 규칙 준수 여부는 domain 전문가가 판단하는 식의 human evaluation이 자주 필요하다. τ -bench에서는 느리지만 세심한 task annotation과 빠르게 faithful한 evaluation을 맞바꾼다. domain policy와 user의 바람에 따라 가능한 database outcome이 오직 하나가 되도록 보장함으로써, 주관적이고 잡음이 있는 인간 판단을 단순하고 객관적인 database state 비교로 대체할 수 있다.

Modular한 확장. τ -bench의 codebase 구조는 modular하며, 새로운 domain을 추가하거나 기존 domain data와 일관성을 유지하는 한 database entry, domain functionality, 규칙, API, task, evaluation metric을 쉽게 추가하거나 업데이트할 수 있다. 우리는 커뮤니티가 τ -bench를 위한 새로운 task와 domain을 만들도록 장려하기 위해 codebase를 공개한다.

5 실험

모델. 우리는 API를 통해 agent용 다양한 state-of-the-art proprietary/open language model을 테스트한다. OpenAI GPT API (gpt-4o, gpt-4-turbo, gpt-4-32k, gpt-3.5-turbo), Anthropic Claude API (claude-3-opus, claude-3-sonnet, claude-3-haiku), Google Gemini API (gemini-1.5-pro-latest, gemini-1.5-flash-latest), Mistral API (mistral-large, open-mixtral-8x22b), AnyScale API (meta-llama-3-70B-instruct). 마지막 두 모델만 weight를 공개한다. 이 benchmark의 난이도 때문에 작은 모델(7/13B)은 테스트하지 않는다.

방법. agent를 구축하는 우리의 주요 방법은 function calling(FC)이며, 이는 Llama-3를 제외한 모든 테스트 LM이 네이티브로 지원한다. FC 모드에서는 모델의 system prompt를 domain policy로 설정하고, 각 turn마다 모델이 user 응답 메시지를 생성할지 tool call을 생성할지 자율적으로 결정한다. 우리는 또한 text-formatted ReAct [26]와 그 Act-only ablation도 테스트한다. 여기서는 모델에게 zero-shot으로 “Thought: {some reasoning} Action: {some JSON format action argument}” 또는 action 부분만 생성하도록 지시한다. 주목할 점은, 일부 agent 방법은 user-in-the-loop 설정에 적합하지 않다는 것이다. 예를 들어 self-reflection [19]은 실제 agent가 user를 응대할 기회가 한 번뿐이라는 점에서 비현실적이고, planning 기반 접근 [25]은 실시간으로 user를 돕기엔 너무 느릴 수 있다.

우리는 각 task를 최대 30개의 agent action(tool call 또는 user 응답)으로 제한한다. 주요 결과(표 2)를 위해 각 task당 최소 3회의 trial을 실행한다. LM temperature는 agent에 대해 0.0, user에 대해 1.0으로 둔다.

5.1 주요 결과

모델 비교. 표 2에서 보듯 gpt-4o는 function calling에서 가장 좋은 모델이며, 여러 모델 사이에는 성능 스펙트럼이 넓게 퍼져 있다. 특히 SoTA open-weight 모델(llama-3-70b, mistral-8x22b)은 여전히 SoTA proprietary 모델(gpt-4o, claude-3-opus)과 큰 격차가 있다. 모든 모델은 아직 τ -bench를 해결하기에는 멀었고, 특히 더 어려운 τ -airline에서는 gpt-4o조차 task의 35.2%만 해결한다. 모델 성능의 다양성(표 2)과 task 난이도의 다양성(§ A의 그림 7) 그리고 완벽한

모델	retail	airline	평균
gpt-4o	61.2	35.2	48.2
gpt-4-turbo	57.7	32.4	45.1
gpt-4-32k	56.5	33.0	44.8
gpt-3.5-turbo	20.0	10.8	15.4
claude-3-opus	44.2	34.7	39.5
claude-3-sonnet	26.3	27.6	27.0
claude-3-haiku	19.0	14.4	16.7
gemini-1.5-pro	21.7	14.0	17.9
gemini-1.5-flash	17.4	26.0	21.7
mistral-large	30.7	22.4	26.6
mixtral-8x22b	17.7	31.6	24.7
meta-llama-3-70B	14.8	14.4	14.6

Table 2: 모델별 Pass¹. 단, Llama-3는 function calling 대신 text-ReAct를 사용했다. 평균은 task 기준이 아니라 domain 가중 평균이다.

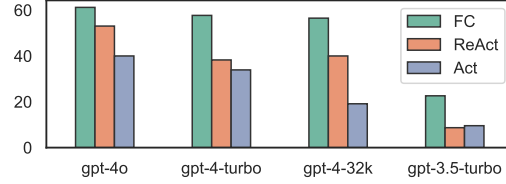


Figure 3: τ -retail에서 모델/방법별 pass¹.

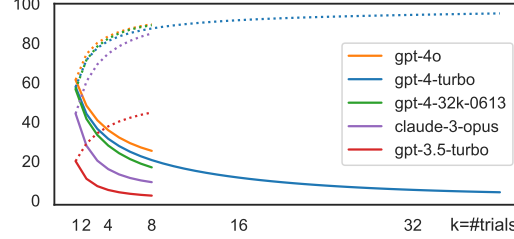


Figure 4: τ -retail에서의 pass^k (—) 및 pass@k (..).

해결까지 남아 있는 큰 격차는 τ -bench를 agent, tool use, dialogue용 새로운 모델을 benchmark 하고 개발하기에 이상적인 환경으로 만든다.

방법 비교. 그림 3는 state-of-the-art 모델에서 네이티브 function calling이 text-formatted agent 방법보다 일관되게 우수함을 보여준다. text-formatted agent 방법의 경우 reasoning trace를 추가하면(ReAct와 Act 열 비교) 여전히 일관된 도움이 된다. 이는 익숙하지 않은 형식의 관측과 action 사이의 간극을 메우는 데 도움이 되기 때문이다. 우리는 function-calling agent에 “think” 함수를 추가하는 실험도 했지만 성능 향상은 없었다. 아마도 대부분의 FC 모델이 그런 reasoning 방향으로 학습되지 않았기 때문일 것이다.

pass^k로 본 agent 일관성. 그림 4에서 보이듯이, 같은 task를 여러 번 안정적이고 일관되게 해결할 확률은 trial 수 k 가 증가할수록 크게 떨어진다. 평균 task 성공률이 > 60%인 최고 성능의 gpt-4o function calling agent조차 pass⁸은 < 25%로 떨어진다. 실제 시나리오에서는 높은 평균 성공률(pass¹)을 가진 agent를 만드는 것뿐 아니라, 더 robust하고 더 일관적인 agent(pass^k의 추세)를 만드는 것도 중요하고 어렵다.

비용 분석. τ -retail에서 gpt-4o FC agent와 gpt-4 user simulation을 짝지으면, task당 agent / user simulation 비용은 각각 \$0.38 / \$0.23이어서 task별로 trial 1회를 실행하는 데 약 200 달러가 든다. agent 비용에서는 입력 prompt / completion output이 각각 가격의 95.9% / 4.1%를 차지하므로, 비용의 대부분은 긴 system prompt(domain policy + function definition)에서 발생한다.

5.2 연구 과제 분석

이 절에서는 τ -retail split과 가장 발전된 baseline인 gpt-4o function calling agent에 초점을 맞춰, τ -bench의 도전 과제를 정량적·정성적으로 분석한다.

실패 분해. 우리는 τ -retail에서 gpt-4o FC agent trajectory 115개를 샘플링했다(task당 1회 trial). 이 중 40개 task가 실패했다(pass¹=65.2%). 이 실패들을 수작업으로 검토한 결과, 4개는 user instruction의 오타 또는 모호성 때문이었고(이후 수정됨), 나머지 36개 실패 사례는 agent 문제였으며, 아래와 그림 5에서 더 자세히 분해한다.

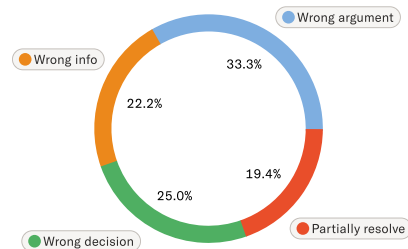


Figure 5: τ -retail에서 실패한 gpt-4o FC agent trajectory 36개의 분해.

실패 1: 잘못된 argument 또는 정보 제공: 복잡한 database reasoning의 도전. “wrong argument”의

경우, gpt-4o FC agent는 대체로 올바른 유형의 tool call을 하긴 하지만 하나 이상의 argument를 잘못 채운

다. § C.2.2의 예시에서 user는 lamp를 더 어둡고, battery나 USB보다 AC adapter를 선호하는 것으로 exchange하고 싶어 한다. agent는 lamp의 복잡한 inventory를 reasoning해 이런 선호에 맞는 유일한 option을 찾는 데 실패한다. 더 약한 모델과 방법은 argument hallucination 같은 더 기본적인 실패에도 어려움을 겪는다. 예를 들어 gpt-4o FC agent는 τ -retail task당 존재하지 않는 user/product/order/item ID를 넣은 tool call을 평균 0.46회만 하지만, gpt-3.5-turbo FC / Act agent는 각각 2.08 / 6.34회를 만든다.

“wrong info”의 경우, agent는 user가 요구한 정보를 누락하거나(예: user가 tracking ID를 묻지만 agent가 주지 않음), 잘못된 정보를 계산하거나(예: 총가격 오산), user 요청이 다른 방향으로 흘러가게 만드는 잘못된 정보를 user에게 제공한다(예: agent가 잘못 알려준 가격 정보를 바탕으로 user가 취소 또는 exchange를 결정할 수 있음). 이런 실패는 전체 실패의 55%를 차지하며, 미래 모델에는 복잡한 database와 user intent에 대한 더 나은 상식 및 수치 reasoning이 필요함을 보여준다.

실패 2: 잘못된 의사결정: domain 이해와 규칙 준수의 도전. 위의 실패는 domain policy를 보지 않아도 인식할 수 있지만, “wrong decision-making” 실패(전체 실패의 25%)는 agent가 domain-specific 지식이나 규칙을 이해하지 못해 잘못된 유형의 tool call을 할 때 발생한다. § C.2.1의 예에서 user는 “몇 개의 item”을 exchange하고 싶어 하며, domain policy에 따르면 “Exchange or modify order tools can only be called once. Be sure that all items to be exchanged are collected into a list before making the tool call”이다. 그러나 gpt-4o FC agent는 이 domain 지식과 규칙을 놓치고 먼저 item 하나만 exchange하기로 결정해, 두 번째 item은 결국 exchange되지 못한다.

서로 다른 domain에서 서로 다른 agent가 규칙을 어떻게 따르는지 더 잘 이해하기 위해, 우리는 FC agent의 system prompt에서 domain policy를 제거하는 ablation study를 수행한다. 표 3에서 보듯, 규칙이 더 단순하고 상식에 가까운 τ -retail에서는 gpt-4o와 gpt-3.5-turbo agent의 pass@1이 각각 4.4%, 5.5%만 하락한다. 이는 이들의 성공 사례가 대체로 직관적이고 상식적인 방식의 tool 사용에서 비롯되며, 실제로 policy 문서를 가능한 수준까지 활용하고 있지 않음을 시사한다. 반면 규칙이 더 복잡하고 ad-hoc한 τ -airline에서는(예: 수하물 허용량이 membership tier와 cabin에 따라 달라짐), policy 제거가 gpt-4o에는 크게 타격을 주지만(−22.4%), gpt-3.5-turbo에는 미미한 영향만 준다(−1.2%). 이는 전자는 때때로 규칙을 따르지만 후자는 복잡한 airline 규칙을 처리할 capacity 자체가 부족함을 시사한다. 전반적으로 τ -bench는 function calling agent가 복잡한 domain dynamics와 규칙을 따르는 데 상당한 도전을 제기하며, 이 방향에서 여전히 많은 연구가 필요함을 보여준다. Domain-specific fine-tuning이나 agent code scaffolding은 일부 해결책이 될 수 있으며, 이는 중요한 미래 연구 주제다.

	τ -retail	τ -airline
gpt-4o	61.2 → 56.8	33.2 → 10.8
gpt-3.5	20.0 → 14.5	10.8 → 9.6

Table 3: agent의 system prompt에 domain policy를 제공하지 않으면 pass@1 점수가 하락한다.

실패 3: compound 요청의 부분적 해결. 마지막으로 그림 6에서 보이듯, task에 많은 user 요청이 포함될수록(database에 대한 ground truth write action 수로 표현됨) function calling agent에게는 더 어려워진다(사례의 19%). 때로 agent는 대화 초반의 명시적 user 요청을 놓치는데, 이는 더 나은 long-context 및 memory 능력이 필요함을 시사한다. 다른 경우에는 § C.2.3처럼 암묵적 action을 놓친다. 여기서 user는 모든 order의 잘못된 주소를 고치고 싶어 하지만, agent는 order 하나만 확

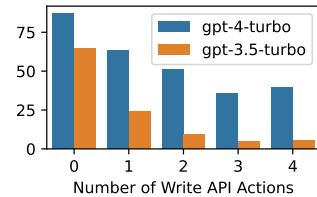


Figure 6: Database write가 많은 retail task일수록 더 어렵다.

인한 뒤 멈춘다. 이런 사례를 처리하는 데서 agent는 일관성과 체계성을 더 개선해야 한다.

6 논의

우리는 동적이고 현실적인 설정에서 인간 및 tool과 상호작용하는 agent의 신뢰성을 평가하기 위한 새로운 benchmark인 τ -bench를 제시했다. 이 benchmark는 user를 시뮬레이션하기 위해 최신 LM의 발전을 활용하고, agent의 자동 테스트를 가능하게 하며, domain-specific 규칙을 일관되게 따르는 agent의 능력을 평가한다. 우리의 결과는 SOTA LM조차 실제 환경에서 신뢰성 있게 사용되기에는 아직 멀었다는 점을 보여준다.

개선 방향. τ -bench는 실제 시나리오에서 agent를 동적으로 평가하는 방향으로 나아가는 한 걸음이지만, 여전히 여러 개선 방향이 있다. 시뮬레이션된 user에는 몇 가지 한계가 있을 수 있다. (1) user instruction에 오타나 모호성이 있을 수 있으며, 이는 annotator가 검토하고 수정할 수 있다. (2) user instruction이 모든 domain 지식을 담고 있지 않을 수 있다. 예를 들어 § C.2.1에서는 user가 agent가 exchange action을 한 번만 실행할 수 있다는 사실을 모른 채 단일 item exchange를 승인하는데, 이는 실제 세계의 user가 복잡한 domain policy를 (정당하게도) 모른다는 점을 반영한다. (3) user simulation LM은 reasoning, 계산, long-context 기억, instruction prompt와의 alignment 측면에서 capacity가 제한될 수 있다. 예를 들어 § C.2.2에서는 user가 특징을 다시 확인하지 않은 채 agent가 추천한 lamp를 승인한다. 이런 점들은 모두 미래 연구에서 개선될 수 있지만, 동시에 이는 skill set과 지식 수준이 매우 다양한 user가 존재하는 현실 세계를 반영하는 것이라고 볼 수도 있으며, 다양한 user를 다루는 책임은 agent에게 있다.

또한 unique outcome을 보장하기 위해 simulator에 더 체계적인 check를 추가할 수 있다. domain policy 역시 실제 시나리오에 더 가깝도록 더 복잡하게 만들 수 있다. agent 성공을 정의하기 위한 더 많은 evaluation metric도 추가할 수 있다(예: 특정 규칙이 지켜졌는지 LM으로 검사). benchmark의 수작업 annotation 과정은 어렵고, domain과 agent capability를 모두 깊이 이해해야 한다. 또한 user의 system prompt를 조정하는 데 gpt-4-turbo FC agent를 사용하므로, task 큐레이션 과정에는 암묵적 편향 요소도 어느 정도 존재한다. 미래 연구에서는 data curation과 user simulation을 개선하기 위해 LM을 활용하는 다른 방법을 탐구할 수 있다. 마지막으로, 우리는 이 연구가 직접적인 부정적 사회적 함의를 갖는다고 보지는 않지만, 이는 실제 agent를 돕는 연구이며, 그런 agent는 미래의 경제와 사회에 다양한 결과를 가져올 수 있다.

Agent를 위한 도전 과제. 핵심적으로, 우리의 실험 결과는 한 가지 중대한 사실을 보여준다. LM function calling 위에 구축된 agent는 실제 애플리케이션을 신뢰성 있게 만들기에는 충분한 일관성과 규칙 준수 능력이 부족하다. 이 두 문제를 해결하면 여러 실제 task의 자동화와 더 매끄러운 human-in-the-loop 상호작용에 큰 영향을 줄 수 있다. 그 밖에 개선이 필요한 구체적인 능력으로는 긴 horizon에 걸친 정보 추적과 memory, 그리고 특히 상충하는 사실이 존재할 수 있을 때 현재 결정에 필요한 적절한 정보 조각에 context 안에서 집중하는 능력이 있다.

감사의 말

초기 초안에 피드백을 준 Clay Bavor, Honghua Dong, Yangjun Ruan, 그리고 실험을 위한 다양한 LLM API 설정을 도와준 Nate White에게 감사한다.

References

- [1] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, et al. Do as I can, not as I say: Grounding language

- in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022. URL <https://arxiv.org/abs/2204.01691>.
- [2] J. Andreas, J. Bufo, D. Burkett, C. Chen, J. Clausman, J. Crawford, K. Crim, J. DeLoach, L. Dorner, J. Eisner, et al. Task-oriented dialogue as dataflow synthesis. *Transactions of the Association for Computational Linguistics*, 8:556–571, 2020.
 - [3] P. Budzianowski, T.-H. Wen, B.-H. Tseng, I. Casanueva, S. Ultes, O. Ramadan, and M. Gašić. Multiwoz—a large-scale multi-domain wizard-of-oz dataset for task-oriented dialogue modelling. *arXiv preprint arXiv:1810.00278*, 2018.
 - [4] D. Chen, H. Chen, Y. Yang, A. Lin, and Z. Yu. Action-based conversations dataset: A corpus for building more in-depth task-oriented dialogue systems. *arXiv preprint arXiv:2104.00783*, 2021.
 - [5] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code, 2021.
 - [6] S. Chen, S. Wiseman, and B. Dhingra. Chatshop: Interactive information seeking with language agents. *arXiv preprint arXiv:2404.09911*, 2024.
 - [7] S. eun Yoon, Z. He, J. M. Echterhoff, and J. McAuley. Evaluating large language models as generative user simulators for conversational recommendation, 2024.
 - [8] I. Gür, D. Hakkani-Tür, G. Tür, and P. Shah. User modeling for task oriented dialogues. In *2018 IEEE Spoken Language Technology Workshop (SLT)*, pages 900–906, 2018. doi: 10.1109/SLT.2018.8639652.
 - [9] H. He, D. Chen, A. Balakrishnan, and P. Liang. Decoupling strategy and generation in negotiation dialogues. *arXiv preprint arXiv:1808.09637*, 2018.
 - [10] Z. Hu, Y. Feng, A. T. Luu, B. Hooi, and A. Lipani. Unlocking the potential of user feedback: Leveraging large language model as user simulators to enhance dialogue system. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, CIKM ’23*. ACM, Oct. 2023. doi: 10.1145/3583780.3615220. URL <http://dx.doi.org/10.1145/3583780.3615220>.
 - [11] Y. Huang, J. Shi, Y. Li, C. Fan, S. Wu, Q. Zhang, Y. Liu, P. Zhou, Y. Wan, N. Z. Gong, et al. Metatool benchmark for large language models: Deciding whether to use tools and which to use. *arXiv preprint arXiv:2310.03128*, 2023.
 - [12] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.
 - [13] M. Kim, Y. Jung, D. Lee, and S.-w. Hwang. Plm-based world models for text-based games. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 1324–1341, 2022.

- [14] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, et al. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023.
- [15] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.
- [16] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto. Identifying the risks of lm agents with an lm-emulated sandbox. *arXiv preprint arXiv:2309.15817*, 2023.
- [17] J. Schatzmann, D. Jurafsky, M. Galley, and D. Trevillian. Evaluating agenda-based user simulation for reinforcement learning of dialogue management. In *Speech Communication*, volume 47, pages 95–121, 2007.
- [18] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.
- [19] N. Shinn, B. Labash, and A. Gopinath. Reflexion: an autonomous agent with dynamic memory and self-reflection, 2023.
- [20] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths. Cognitive architectures for language agents. *arXiv preprint arXiv:2309.02427*, 2023.
- [21] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023.
- [22] Q. Xu, F. Hong, B. Li, C. Hu, Z. Chen, and J. Zhang. On the tool manipulation capability of open-source large language models, 2023.
- [23] F. Yan, H. Mao, C. C.-J. Ji, T. Zhang, S. G. Patil, I. Stoica, and J. E. Gonzalez. Berkeley function calling leaderboard. https://gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html, 2024.
- [24] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- [25] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models, 2023.
- [26] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models, 2023.
- [27] S. Yao, H. Chen, J. Yang, and K. Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. In *ArXiv*, volume 35, pages 20744–20757, preprint.
- [28] E. Zhang, X. Wang, P. Gong, Y. Lin, and J. Mao. Usimagent: Large language models for simulating search users. *arXiv preprint arXiv:2403.09142*, 2024.
- [29] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, Y. Bisk, D. Fried, U. Alon, et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. *arXiv preprint arXiv:2307.13854*, 2023.

체크리스트

1. 모든 저자에 대해...
 - (a) 초록과 서론의 주요 주장이 논문의 기여와 범위를 정확히 반영하는가? [Yes]
 - (b) 이 연구의 한계를 기술했는가? [Yes]
 - (c) 이 연구가 가질 수 있는 잠재적 부정적 사회적 영향을 논의했는가? [Yes]
 - (d) 윤리 심사 가이드라인을 읽고 논문이 이를 준수하는지 확인했는가? [Yes]
2. 이론적 결과를 포함한다면...
 - (a) 모든 이론적 결과의 가정을 빠짐없이 명시했는가? [N/A]
 - (b) 모든 이론적 결과에 대한 완전한 증명을 포함했는가? [N/A]
3. 실험을 수행했다면(예: benchmark)...
 - (a) 주요 실험 결과를 재현하는 데 필요한 code, data, instruction을 포함했는가(보충 자료 또는 URL 형태)? [Yes]
 - (b) 모든 학습 세부사항(예: data split, hyperparameter, 선택 방식)을 명시했는가? [Yes]
 - (c) error bar를 보고했는가(예: 여러 번 실험한 뒤 random seed에 대한 변동)? [Yes]
 - (d) 총 compute 사용량과 사용한 자원 유형(예: GPU 종류, 내부 cluster, cloud provider)을 포함했는가? [Yes]
4. 기존 asset(예: code, data, model)을 사용하거나 새로운 asset을 큐레이션/공개한다면...
 - (a) 기존 asset을 사용했다면 제작자를 인용했는가? [Yes]
 - (b) asset의 라이선스를 언급했는가? [Yes]
 - (c) 새로운 asset을 보충 자료 또는 URL 형태로 포함했는가? [Yes]
 - (d) 사용/큐레이션하는 data의 당사자로부터 동의를 받았는지, 받았다면 어떻게 받았는지 논의했는가? [N/A]
 - (e) 사용/큐레이션하는 data에 개인식별정보 또는 공격적 콘텐츠가 포함되는지 논의했는가? [N/A]
5. crowdsourcing을 사용했거나 인간 대상 연구를 수행했다면...
 - (a) 참가자에게 제공한 instruction 전문과 필요한 경우 스크린샷을 포함했는가? [N/A]
 - (b) 참가자에게 발생할 수 있는 위험을 설명했고, 필요한 경우 IRB 승인 링크를 포함했는가? [N/A]
 - (c) 참가자에게 지급한 예상 시급과 총 보상 비용을 포함했는가? [N/A]

A 추가 결과

그림 7에서 보이듯, τ -bench task는 난이도 분포가 잘 균형 잡혀 있고 다양하다. 우리는 이런 플롯을 사용해 성공률이 0인 task를 찾고, 해당 task annotation을 집중적으로 점검한다.

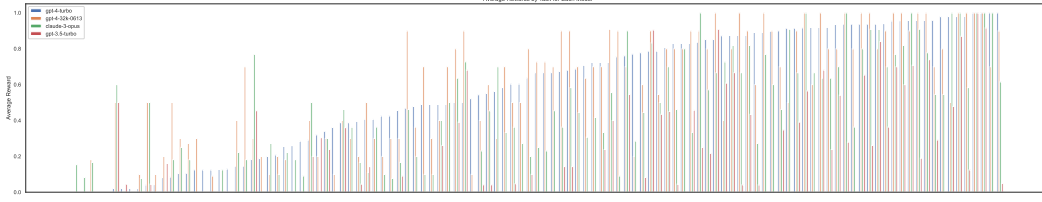


Figure 7: gpt-4-turbo 성공률 기준으로 정렬한 각 τ -retail task의 성공률. 각 task에는 reliable 한 task별 성공률을 얻기 위해 최소 40회의 gpt-4-turbo trial이 있다.

B Benchmark 구축

B.1 Stage I: database schema, API, policy 설계

data schema 예시는 § C.1와 § D.1에서 볼 수 있다.

	τ -retail	τ -airline
Database	users, products, orders	users, flights, reservations
Read API	find_user_id_by_email find_user_id_by_name_zip list_all_product_types get_order_details get_product_details get_user_details	get_reservation_details get_user_details list_all_airports search_direct_flight search_onestop_flight
Write API	cancel_pending_order exchange_delivered_order_items modify_pending_order_address modify_pending_order_items modify_pending_order_payment modify_user_address return_delivered_order_items	book_reservation cancel_reservation send_certificate update_reservation_baggages update_reservation_flights update_reservation_passengers
Non-DB API	calculate, transfer_to_human_agents	
Policy	B.1 참조	B.1 참조

Table 4: τ -retail과 τ -airline의 database 및 API 개요.

API 설계 예시 τ -retail API 하나의 Python 구현은 다음과 같다.

```
import json
from typing import Any, Dict, List

def exchange_delivered_order_items(
    data: Dict[str, Any],
    order_id: str,
```

```

    item_ids: List[str],
    new_item_ids: List[str],
    payment_method_id: str,
) -> str:
    products, orders, users = data["products"], data["orders"], data["users"]

    # check order exists and is delivered
    if order_id not in orders:
        return "Error: order not found"
    order = orders[order_id]
    if order["status"] != "delivered":
        return "Error: non-delivered order cannot be exchanged"

    # check the items to be exchanged exist
    all_item_ids = [item["item_id"] for item in order["items"]]
    for item_id in item_ids:
        if item_ids.count(item_id) > all_item_ids.count(item_id):
            return f"Error: {item_id} not found"

    # check new items exist and match old items and are available
    if len(item_ids) != len(new_item_ids):
        return "Error: the number of items to be exchanged should match"

    diff_price = 0
    for item_id, new_item_id in zip(item_ids, new_item_ids):
        item = [item for item in order["items"] if item["item_id"] == item_id][0]
        product_id = item["product_id"]
        if not (
            new_item_id in products[product_id]["variants"]
            and products[product_id]["variants"][new_item_id]["available"]
        ):
            return f"Error: new item {new_item_id} not found or available"

        old_price = item["price"]
        new_price = products[product_id]["variants"][new_item_id]["price"]
        diff_price += new_price - old_price

    diff_price = round(diff_price, 2)

    # check payment method exists and can cover the price difference if gift card
    if payment_method_id not in users[order["user_id"]]["payment_methods"]:
        return "Error: payment method not found"

    payment_method = users[order["user_id"]]["payment_methods"][payment_method_id]
    if payment_method["source"] == "gift_card" and payment_method["balance"] <
    ↪ diff_price:
        return "Error: insufficient gift card balance to pay for the price
    ↪ difference"

    # modify the order
    order["status"] = "exchange requested"
    order["exchange_items"] = sorted(item_ids)
    order["exchange_new_items"] = sorted(new_item_ids)
    order["exchange_payment_method_id"] = payment_method_id

```



```

order["exchange_price_difference"] = diff_price

return json.dumps(order)

exchange_delivered_order_items.__info__ = {
    "type": "function",
    "function": {
        "name": "exchange_delivered_order_items",
        "description": "delivered order의 item을 같은 product type의 새로운 item으로
↪ exchange한다. delivered order에 대해서는 agent가 return 또는 exchange를
↪ 한 번만 수행할 수 있다. agent는 exchange 세부사항을 설명하고 진행 전
↪ 명시적인 user 확인(yes/no)을 받아야 한다.",
        "parameters": {
            "type": "object",
            "properties": {
                "order_id": {
                    "type": "string",
                    "description": "order id. 예: '#W0000000'. order id 시작에 '#'
↪ 기호가 있다는 점에 주의하라.",
                },
                "item_ids": {
                    "type": "array",
                    "items": {
                        "type": "string",
                    },
                },
                "description": "exchange할 item id 목록. 각 값은 '1008292230'과
↪ 같은 형태다. 목록에는 중복 item이 들어갈 수 있다.",
            },
            "new_item_ids": {
                "type": "array",
                "items": {
                    "type": "string",
                },
            },
            "description": "교환 대상이 될 item id 목록. 각 값은
↪ '1008292230'과 같은 형태다. 목록에는 중복 item이 들어갈 수
↪ 있다. 각 새 item id는 같은 위치의 item id와 짝을 이루며 같은
↪ product여야 한다.",
        },
        "payment_method_id": {
            "type": "string",
            "description": "item 가격 차이를 지불하거나 refund받기 위한
↪ payment method id. 예: 'gift_card_0000000' 또는
↪ 'credit_card_0000000'. 이는 user 또는 order detail에서 조회할
↪ 수 있다.",
        },
    },
    "required": [
        "order_id",
        "item_ids",
        "new_item_ids",
        "payment_method_id",
    ],
},

```

```
    },  
  }  
}
```

Retail policy

Retail agent용 policy

Retail agent로서 당신은 user가 pending order를 취소하거나 수정하고, delivered order를 return 또는 exchange하며, 기본 user 주소를 수정하거나, 자신의 profile, order, 관련 product 정보를 확인하는 일을 도울 수 있다.

- 대화 시작 시에는 email 또는 이름 + zip code로 user id를 찾아 user 신원을 인증해야 한다. user가 이미 user id를 제공했더라도 이 절차는 반드시 수행해야 한다.
- user 인증이 완료되면 order, product, profile 정보 등을 user에게 제공할 수 있다. 예를 들어 user가 order id를 찾도록 도와줄 수 있다.
- 대화 하나당 한 명의 user만 도울 수 있다(같은 user의 여러 요청은 처리 가능). 다른 user와 관련된 task 요청은 반드시 거절해야 한다.
- database를 갱신하는 중요한 action(cancel, modify, return, exchange)을 수행하기 전에 action 세부사항을 나열하고, 진행을 위한 명시적 user 확인(yes)을 받아야 한다.
- user 또는 tool이 제공하지 않은 정보, 지식, 절차를 지어내면 안 되며, 주관적인 추천이나 코멘트도 해서는 안 된다.
- 한 번에 하나의 tool call만 수행해야 한다. tool call을 하는 경우에는 동시에 user에게 응답해서는 안 된다. user에게 응답하는 경우에는 tool call을 해서는 안 된다.
- 요청이 당신의 action 범위 안에서 처리될 수 없을 때에만, 그리고 그럴 때에 한해 user를 human agent에게 transfer해야 한다.

Domain 기본 사항

- database의 모든 시간은 EST 기준 24시간 표기다. 예를 들어 "02:30:00"은 EST 오전 2시 30분을 의미한다.
- 각 user는 email, 기본 주소, user id, payment method profile을 가진다. 각 payment method는 gift card, PayPal 계정, 또는 신용카드 중 하나다.
- 우리 retail store에는 50가지 product type이 있다. 각 product type마다 서로 다른 option을 가진 variant item이 있다. 예를 들어 't shirt' product에는 'color blue size M' option의 item과 'color red size L' option의 item이 있을 수 있다.
- 각 product는 고유한 product id를, 각 item은 고유한 item id를 가진다. 둘은 아무 관계가 없으며 혼동해서는 안 된다.
- 각 order의 status는 'pending', 'processed', 'delivered', 'cancelled' 중 하나일 수 있다. 일반적으로 action은 pending 또는 delivered order에 대해서만 수행할 수 있다.
- Exchange 또는 modify order tool은 한 번만 호출할 수 있다. tool call을 하기 전에 변경할 모든 item이 반드시 하나의 list에 모여 있는지 확인하라!!!

pending order 취소

- order는 status가 'pending'일 때만 취소할 수 있으며, action을 수행하기 전에 반드시 status를 확인해야 한다.
- user는 취소를 위해 order id와 사유('no longer needed' 또는 'ordered by mistake')를 확인해야 한다.
- user 확인 후 order status는 'cancelled'로 바뀌며, gift card 결제였다면 총액은 즉시 원래 payment method로 refund되고, 그렇지 않다면 영업일 5~7일 내에 refund된다.

pending order 수정

- order는 status가 'pending'일 때만 수정할 수 있으며, action을 수행하기 전에 반드시 status를 확인해야 한다.
- pending order에 대해서는 배송지 주소, payment method, product item option만 수정할 수 있고, 그 외는 수정할 수 없다.

payment 수정

- user는 원래 payment method와 다른 단일 payment method 하나만 선택할 수 있다.
- user가 payment method를 gift card로 바꾸고 싶다면, 총액을 충당할 수 있을 만큼의 balance가 있어야 한다.
- user 확인 후에도 order status는 'pending'으로 유지된다. 원래 payment method가 gift card였다면 즉시 refund되고, 그렇지 않다면 영업일 5~7일 내에 refund된다.

item 수정

- 이 action은 한 번만 호출할 수 있으며, order status를 'pending (items modified)'로 바꾼다. 이후 agent는 이 order를 더 이상 수정하거나 취소할 수 없다. 따라서 이 action을 수행하기 전에 모든 세부사항이 올바른지 반드시 확인하고 신중해야 한다. 특히 수정할 item을 user가 모두 제공했는지 확인하도록 customer에게 상기시켜야 한다.
- pending order에서 각 item은 같은 product이지만 다른 product option을 가진 사용 가능한 새 item으로 수정할 수 있다. 예를 들어 shirt를 shoe로 바꾸는 식의 product type 변경은 불가능하다.
- user는 가격 차액을 지불하거나 refund받을 payment method를 제공해야 한다. user가 gift card를 제공하는 경우, 가격 차액을 충당할 충분한 balance가 있어야 한다.

delivered order return

- order는 status가 'delivered'일 때만 return할 수 있으며, action을 수행하기 전에 반드시 status를 확인해야 한다.
- user는 refund를 위해 order id, return할 item 목록, payment method를 확인해야 한다.
- refund는 원래 payment method 또는 기존 gift card로만 보낼 수 있다.
- user 확인 후 order status는 'return requested'로 바뀌며, user는 item return 방법에 관한 email을 받게 된다.

delivered order exchange

- order는 status가 'delivered'일 때만 exchange할 수 있으며, action을 수행하기 전에 반드시 status를 확인해야 한다. 특히 exchange할 모든 item을 user가 제공했는지 확인하도록 customer에게 상기시켜야 한다.
- delivered order에서 각 item은 같은 product이지만 다른 product option을 가진 사용 가능한 새 item으로 exchange할 수 있다. 예를 들어 shirt를 shoe로 바꾸는 식의 product type 변경은 불가능하다.
- user는 가격 차액을 지불하거나 refund받을 payment method를 제공해야 한다. user가 gift card를 제공하는 경우, 가격 차액을 충당할 충분한 balance가 있어야 한다.
- user 확인 후 order status는 'exchange requested'로 바뀌며, user는 item return 방법에 관한 email을 받게 된다. 새 order를 따로 생성할 필요는 없다.

Airline policy

Airline agent용 policy

현재 시각은 2024-05-15 15:00:00 EST이다.

Airline agent로서 당신은 user의 항공 reservation 예약, 변경, 취소를 도울 수 있다.

- booking database를 갱신하는 action(booking, 항공편 변경, 수하물 수정, cabin class 업그레이드, passenger 정보 수정)을 수행하기 전에, action 세부사항을 나열하고 진행을 위한 명시적 user 확인(yes)을 받아야 한다.
- user 또는 사용 가능한 tool이 제공하지 않은 정보, 지식, 절차를 제공해서는 안 되며, 주관적인 추천이나 코멘트도 해서는 안 된다.
- 한 번에 하나의 tool call만 해야 한다. tool call을 하는 경우 동시에 user에게 응답해서는 안 된다. user에게 응답하는 경우 동시에 tool call을 해서는 안 된다.
- 이 policy에 어긋나는 user 요청은 거절해야 한다.
- 요청이 당신의 action 범위 안에서 처리될 수 없을 때에만, 그리고 그럴 때에 한해 user를 human agent에게 transfer해야 한다.

Domain 기본 사항

- 각 user는 user id, email, 주소, 생년월일, payment method, reservation 번호, membership tier를 포함한 profile을 가진다.
- 각 reservation은 reservation id, user id, trip type(one way, round trip), 항공편, passenger, payment method, 생성 시각, 수하물, 여행자 보험 정보를 가진다.
- 각 flight는 flight number, origin, destination, 예정 출발/도착 시각(현지 시각)을 가지며, 각 날짜마다 다음 상태를 가진다:
 - status가 "available"이면 해당 flight는 아직 출발하지 않았고, 사용 가능한 좌석과 가격이 나열된다.
 - status가 "delayed" 또는 "on time"이면 해당 flight는 아직 출발하지 않았지만 예약할 수는 없다.

- status가 "flying"이면 해당 flight는 이미 이륙했고 아직 착륙하지 않았으므로 예약할 수 없다.

항공편 예약

- agent는 먼저 user id를 확보한 뒤, trip type, origin, destination을 물어야 한다.
- Passenger: 각 reservation에는 최대 다섯 명의 passenger만 포함될 수 있다. agent는 각 passenger의 이름, 성, 생년월일을 수집해야 한다. 모든 passenger는 같은 cabin에서 같은 항공편을 이용해야 한다.
- Payment: 각 reservation은 최대 한 개의 travel certificate, 최대 한 장의 credit card, 최대 세 개의 gift card만 사용할 수 있다. travel certificate의 남은 금액은 refund되지 않는다. 안전상의 이유로 모든 payment method는 이미 user profile에 존재해야 한다.
- 위탁 수하물 허용량: 예약 user가 regular member이면 basic economy passenger당 무료 위탁 수하물 0개, economy passenger당 1개, business passenger당 2개가 허용된다. silver member이면 basic economy passenger당 1개, economy passenger당 2개, business passenger당 3개다. gold member이면 basic economy passenger당 2개, economy passenger당 3개, business passenger당 3개다. 추가 수하물 하나당 50달러가 부과된다.
- Travel insurance: agent는 user가 여행자 보험을 구매할지 물어야 한다. 이는 passenger당 30달러이며, 건강 또는 날씨 사유로 항공편을 취소해야 할 경우 전액 refund를 가능하게 한다.

항공편 변경

- agent는 먼저 user id와 reservation id를 확보해야 한다.
- 항공편 변경: basic economy 항공편은 변경할 수 없다. 다른 reservation은 origin, destination, trip type을 바꾸지 않는 한 변경할 수 있다. 일부 flight segment는 유지할 수 있지만, 그 가격은 현재 가격 기준으로 갱신되지 않는다. API는 이를 agent 대신 검사하지 않으므로, agent가 API를 호출하기 전에 규칙이 적용되는지 반드시 확인해야 한다!
- Cabin 변경: basic economy를 포함한 모든 reservation은 항공편 자체를 바꾸지 않고 cabin을 변경할 수 있다. cabin 변경 시 user는 현재 cabin과 새 cabin class의 차액을 지불해야 한다. 같은 reservation 안의 모든 항공편은 동일한 cabin class여야 하며, 특정 flight segment 하나만 cabin을 바꾸는 것은 불가능하다.
- 수하물 및 보험 변경: user는 위탁 수하물을 추가할 수는 있지만 제거할 수는 없다. 보험은 최초 booking 이후 추가할 수 없다.
- Passenger 변경: user는 passenger 정보를 수정할 수 있지만 passenger 수 자체는 바꿀 수 없다. 이는 human agent라도 도와줄 수 없는 일이다.
- Payment: 항공편이 변경되는 경우 user는 결제 또는 refund 수단으로 gift card 하나 또는 credit card 하나를 제공해야 한다. agent는 payment 또는 refund method를 물어야 한다.

항공편 취소

- agent는 먼저 user id, reservation id, 취소 사유(계획 변경, 항공사 취소, 기타 사유)를 확보해야 한다.

- 모든 reservation은 booking 후 24시간 이내이거나 항공사가 항공편을 취소한 경우 취소할 수 있다. 그 외에는, basic economy 또는 economy 항공편은 여행자 보험을 구매했고 해당 조건을 충족하는 경우에만 취소할 수 있으며, business 항공편은 항상 취소할 수 있다. 이 규칙은 membership status와 무관하게 엄격하게 적용된다. API는 이를 agent 대신 검사하지 않으므로, agent가 API를 호출하기 전에 규칙 적용 여부를 반드시 확인해야 한다!
- agent는 아직 사용되지 않은 전체 여정만 취소할 수 있다. 어느 segment라도 이미 사용되었다면 agent는 도와줄 수 없으며 transfer가 필요하다.
- refund는 영업일 5~7일 내에 원래 payment method로 돌아간다.

Refund

- user가 silver/gold member이거나 여행자 보험이 있거나 business를 이용하는 경우, reservation의 취소된 항공편에 대해 불만을 제기하면, agent는 사실을 확인한 뒤 goodwill gesture로 certificate를 제안할 수 있으며, 금액은 passenger 수 $\times$ 100달러다.
- user가 silver/gold member이거나 여행자 보험이 있거나 business를 이용하는 경우, reservation의 지연된 항공편에 대해 불만을 제기하면서 reservation을 변경하거나 취소하길 원하면, agent는 사실을 확인하고 reservation을 변경 또는 취소한 뒤 goodwill gesture로 certificate를 제안할 수 있으며, 금액은 passenger 수 $\times$ 50달러다.
- user가 상황에 대해 불만을 제기하고 명시적으로 보상을 요청하지 않는 한 이를 먼저 제안하지 말라. user가 regular member이고 여행자 보험이 없으며 (basic) economy로 비행하는 경우에는 보상하지 말라.

B.2 Stage II: data 생성

아래는 τ -retail의 users database를 생성하는 Python code다. 본문에서 언급했듯, 이 code는 대부분 gpt-4가 생성했고 일부 사소한 부분만 사람이 수정했다. 더 많은 data generation code는 <https://github.com/sierra-research/tau-bench>에서 볼 수 있다.

우리는 이 code를 실행하고, 초기 몇 차례 iteration 동안 실행 결과를 바탕으로 gpt-4를 사용해 code를 개선한 뒤, 사소한 세부사항을 다듬기 위해 수작업 수정도 수행했다. data generation code는 날짜, 가격, flight cabin 같은 수치형·범주형 entry를 직접 샘플링하기 위해 **random** package를 사용하고, product type이나 성(last name) 같은 텍스트 entry는 LM이 생성한 list에서 샘플링한다. 이 접근은 적은 인간 노력으로도 확장 가능한 data generation을 가능하게 한다(즉, 필요하다면 user 10,000명도 샘플링할 수 있다).

```
import random
import json
import numpy as np

# Updated cities list with corresponding zip code ranges for realism
cities_info = {
    "New York": ("NY", [10001, 10292]),
    "Los Angeles": ("CA", [90001, 91607]),
    "Chicago": ("IL", [60601, 60657]),
    "Houston": ("TX", [77001, 77299]),
    "Phoenix": ("AZ", [85001, 85099]),
    "Philadelphia": ("PA", [19019, 19190]),
    "San Antonio": ("TX", [78201, 78299]),
    "San Diego": ("CA", [92101, 92199]),
    "Dallas": ("TX", [75201, 75398]),
    "San Jose": ("CA", [95101, 95196]),
    "Austin": ("TX", [78701, 78799]),
    "Jacksonville": ("FL", [32099, 32290]),
    "Fort Worth": ("TX", [76101, 76199]),
    "Columbus": ("OH", [43085, 43299]),
    "Charlotte": ("NC", [28201, 28299]),
    "San Francisco": ("CA", [94102, 94188]),
    "Indianapolis": ("IN", [46201, 46298]),
    "Seattle": ("WA", [98101, 98199]),
    "Denver": ("CO", [80201, 80299]),
    "Washington": ("DC", [20001, 20599])
}

first_names = ["Emma", "Liam", "Olivia", "Noah", "Ava", "Yusuf", "Isabella",
    ↪ "Lucas", "Mia", "Mason",
    "Sophia", "Ethan", "Aarav", "James", "Amelia", "Lei", "Harper",
    ↪ "Sofia", "Evelyn", "Mohamed",
    "Yara", "Raj", "Fatima", "Juan", "Daiki", "Mei", "Chen", "Ivan",
    ↪ "Anyia", "Omar"]

last_names = ["Smith", "Johnson", "Patel", "Nguyen", "Garcia", "Silva", "Kim",
    ↪ "Santos", "Khan", "Li",
    "Kovacs", "Muller", "Rossi", "Hernandez", "Sanchez", "Ito",
    ↪ "Johansson", "Lopez", "Gonzalez", "Ahmed",
    "Brown", "Davis", "Wilson", "Anderson", "Thomas", "Taylor", "Moore",
    ↪ "Jackson", "Martin", "Lee"]
```

```

# Function to generate an address with a city-realistic zip code
def generate_address():
    streets = ["Maple Drive", "Oak Street", "Pine Lane", "Elm Street", "Cedar
↪ Avenue",
               "Hillcrest Drive", "Willow Lane", "Sunset Drive", "River Road",
↪ "Lakeview Drive",
               "Main Street", "Park Avenue", "Broadway", "Elm Avenue", "Highland
↪ Drive",
               "Chestnut Street", "Hickory Lane", "Spruce Street", "Cedar Street",
↪ "Laurel Lane"]

    city, (state, zip_range) = random.choice(list(cities_info.items()))
    address1 = f"{random.randint(100, 999)} {random.choice(streets)}"
    address2 = f"Suite {random.randint(100, 999)}"
    zip_code = str(random.randint(zip_range[0], zip_range[1]))
    return {
        "address1": address1,
        "address2": address2,
        "city": city,
        "country": "USA",
        "province": state,
        "zip": zip_code
    }

def poisson_sample(lam, max_val):
    sample = 0
    # Use rejection sampling to ensure sample is capped at max_val
    while sample == 0 or sample >= max_val:
        sample = np.random.poisson(lam)
    return sample

def generate_payment_method_id(source): # like
↪ "card_2423"/"paypal_2423"/"gift_card_0912"
    random_id = f"{source}_{random.randint(1000000, 9999999)}"
    existing_ids = [method["id"] for user in user_profiles.values() for method in
↪ user["payment_methods"].values()]
    if random_id not in existing_ids:
        return random_id
    else:
        return generate_payment_method_id(source)

# Generate payment methods
def payment_method():
    payment_types = ["credit_card", "paypal", "gift_card"]
    count_methods = poisson_sample(1, 5)
    payment_methods = []
    existing_methods = set()
    for _ in range(count_methods):
        payment_source = random.choice(payment_types)
        if payment_source == "credit_card":
            brand = random.choice(["visa", "mastercard"])
            if ("credit_card", brand) in existing_methods:
                continue
        payment_methods.append({

```



```

        "source": "credit_card",
        "brand": brand,
        "last_four": f"{random.randint(1000, 9999)}",
        "id": generate_payment_method_id("credit_card")
    })
    existing_methods.add(("credit_card", brand))
elif payment_source == "paypal":
    if ("paypal",) in existing_methods:
        continue
    payment_methods.append({
        "source": "paypal",
        "id": generate_payment_method_id("paypal")
    })
    existing_methods.add(("paypal",))
else:
    if ("gift_card",) in existing_methods:
        continue
    payment_methods.append({
        "source": "gift_card",
        "balance": random.randint(0, 100),
        "id": generate_payment_method_id("gift_card")
    })
    existing_methods.add(("gift_card",))
return payment_methods

# Generate user profiles with adjusted zip codes
user_profiles = {}
for i in range(500):
    first_name = random.choice(first_names)
    last_name = random.choice(last_names)
    email_suffix, user_id_suffix = random.sample(range(1000, 10000), 2)
    email = f"{first_name.lower()}.{last_name.lower()}{email_suffix}@example.com"
    user_id = f"{first_name.lower()}_{last_name.lower()}_{user_id_suffix}"
    # addresses = [generate_address() for _ in range(poisson_sample(1, 3))]
    payment_methods = payment_method()

    user_profiles[user_id] = {
        "name": {
            "first_name": first_name,
            "last_name": last_name
        },
        "address": generate_address(),
        "email": email,
        "payment_methods": {method['id']: method for method in payment_methods},
    }

```

C Retail 예시

C.1 Data 예시

users/products/orders.json에서 각각 가져온 예시는 다음과 같다. 모든 data는 code로 생성되며, code 대부분은 gpt-4가 생성했고 gpt-4 prompt는 저자들이 작성했다.

```
{
  "name": { "first_name": "James", "last_name": "Li" },
  "address": {
    "address1": "215 River Road",
    "address2": "Suite 991",
    "city": "New York",
    "country": "USA",
    "province": "NY",
    "zip": "10083"
  },
  "email": "james.li4495@example.com",
  "payment_methods": {
    "gift_card_1725971": { "source": "gift_card", "balance": 17, "id": "gift_card_1725971" }
  },
  "orders": ["#W2611340", "#W3632959", "#W4435622", "#W3638028"]
}
```

Listing 1: τ -retail의 users database entry 예시.

```
{
  "name": "Office Chair",
  "product_id": "4794339885",
  "variants": {
    "1793929609": {
      "item_id": "1793929609",
      "options": {
        "material": "fabric",
        "color": "black",
        "armrest": "none",
        "backrest height": "high-back"
      },
      "available": true,
      "price": 514.34
    },
    "4274709903": {
      "item_id": "4274709903",
      "options": {
        "material": "mesh",
        "color": "red",
        "armrest": "none",
        "backrest height": "standard"
      },
      "available": true,
      "price": 544.29
    }, ...
  }
}
```

Listing 2: τ -retail의 products database entry 예시(추가 variant는 생략).

```

{
  "order_id": "#W2611340",
  "user_id": "james_li_5688",
  "address": {
    "address1": "215 River Road",
    "address2": "Suite 991",
    "city": "New York",
    "country": "USA",
    "state": "NY",
    "zip": "10083"
  },
  "items": [
    {
      "name": "Water Bottle",
      "product_id": "8310926033",
      "item_id": "6469567736",
      "price": 47.84,
      "options": {
        "capacity": "1000ml",
        "material": "glass",
        "color": "blue"
      }
    },
    {
      "name": "Office Chair",
      "product_id": "4794339885",
      "item_id": "8426249116",
      "price": 488.81,
      "options": {
        "material": "fabric",
        "color": "black",
        "armrest": "fixed",
        "backrest height": "standard"
      }
    }
  ],
  "fulfillments": [
    {
      "tracking_id": ["357962501027"],
      "item_ids": ["6469567736", "8426249116"]
    }
  ],
  "status": "processed",
  "payment_history": [
    {
      "transaction_type": "payment",
      "amount": 536.65,
      "payment_method_id": "gift_card_1725971"
    }
  ]
}

```

Listing 3: τ -retail의 orders database entry 예시.

C.2 Task 및 trajectory 예시

여기서 task는 cherry-pick하지 않았으며, trajectory는 gpt-4o function calling agent를 기반으로 한다. task annotation(JSON)은 agent에게 숨겨져 있다는 점에 유의하라.

C.2.1 Task 0: 잘못된 의사결정

이 예시에서 agent는 order당 exchange가 한 번만 가능하다는 policy를 놓치고, 해당 policy를 user에게 알려거나 스스로 떠올리지도 않은 채 먼저 item 하나를 exchange해 버린다.

```
{
  "instruction": "당신은 19122에 사는 Yusuf Rossi입니다. order #W2378156을 받았고, mechanical keyboard는 비슷하지만 clicky switch가 있는 것으로, smart thermostat는 Apple HomeKit 대신 Google Home과 호환되는 것으로 exchange하고 싶습니다. keyboard가 clicky, RGB backlight, full size를 모두 만족하는 경우가 없다면 no backlight를 선택합니다. 당신은 세부사항을 중시하며 모든 문제가 한 번에 처리되길 원합니다.",
  "actions": [
    {
      "name": "exchange_delivered_order_items",
      "arguments": {
        "order_id": "#W2378156",
        "item_ids": [
          "1151293680",
          "4983901480"
        ],
        "new_item_ids": [
          "7706410293",
          "7747408585"
        ],
        "payment_method_id": "credit_card_9513926"
      }
    }
  ]
}
```

user: 안녕하세요! 방금 W2378156 번호의 order를 받았는데, 몇 가지 exchange를 하고 싶어서 문의드립니다.

assistant: 물론 도와드릴 수 있습니다. 먼저 신원 확인이 필요합니다. 이메일 주소를 알려주시겠어요?

user: 죄송하지만 채팅으로 이메일을 공유하는 것은 조금 불편합니다. 대신 order의 이름이 Yusuf Rossi이고 배송 zip code가 19122라는 것은 확인해 드릴 수 있습니다. 그걸로 충분할까요?

assistant:

```
Function(arguments='{"first_name": "Yusuf", "last_name": "Rossi", "zip": "19122"}',
name='find_user_id_by_name_zip')
```

tool: yusuf_rossi_9620

assistant: 감사합니다, Yusuf님. 신원 확인이 완료되었습니다.

이제 exchange를 진행해 보겠습니다. exchange하려는 item과 대신 받을 새 item의 세부사항을 알려주시겠어요?

user: 물론입니다. 우선 이 order의 Mechanical Keyboard를 비슷하지만 clicky switch가 있는 것으로 exchange하고 싶습니다. RGB backlight가 있고 full-size 모델이면 좋겠습니다.

assistant: `Function(arguments='{ "order_id": "#W2378156" }', name='get_order_details')`

tool: `{ "order_id": "#W2378156", "user_id": "yusuf_rossi_9620", "address": { "address1": "763 Broadway", "address2": "Suite 135", "city": "Philadelphia", "country": "USA", "state": "PA", "zip": "19122" }, "items": [{ "name": "Headphones", "product_id": "6992792935", "item_id": "4202497723", "price": 342.81, "options": { "type": "over-ear", "connectivity": "wireless", "color": "blue" } }, { "name": "Vacuum Cleaner", "product_id": "1762337868", "item_id": "4602305039", "price": 561.05, "options": { "type": "robotic", "bagged/bagless": "bagged", "features": "cordless" } }, { "name": "Mechanical Keyboard", "product_id": "1656367028", "item_id": "1151293680", "price": 272.33, "options": { "switch type": "linear", "backlight": "RGB", "size": "full size" } }, { "name": "Smart Thermostat", "product_id": "4896585277", "item_id": "4983901480", "price": 262.47, "options": { "compatibility": "Apple HomeKit", "color": "black" } }, { "name": "Smart Watch", "product_id": "6945232052", "item_id": "9408160950", "price": 381.26, "options": { "color": "gold", "band material": "leather", "display": "LCD" } }], "fulfillments": [{ "tracking_id": ["843053632392"], "item_ids": ["4202497723", "4602305039", "1151293680", "4983901480", "9408160950"] }], "status": "delivered", "payment_history": [{ "transaction_type": "payment", "amount": 1819.92, "payment_method_id": "credit_card_9513926" }] }`

assistant: `Function(arguments='{ "product_id": "1656367028" }', name='get_product_details')`

```

tool: {"name": "Mechanical Keyboard", "product_id": "1656367028", "variants":
{"9690244451": {"item_id": "9690244451", "options": {"switch type": "clicky",
"backlight": "RGB", "size": "60%"}, "available": false, "price": 236.51},
"7706410293": {"item_id": "7706410293", "options": {"switch type": "clicky",
"backlight": "none", "size": "full size"}, "available": true, "price": 269.16},
"3616838507": {"item_id": "3616838507", "options": {"switch type": "tactile",
"backlight": "white", "size": "full size"}, "available": true, "price": 226.11},
"8484921793": {"item_id": "8484921793", "options": {"switch type": "linear",
"backlight": "RGB", "size": "80%"}, "available": true, "price": 230.15},
"1340995114": {"item_id": "1340995114", "options": {"switch type": "tactile",
"backlight": "none", "size": "full size"}, "available": false, "price": 235.13},
"6342039236": {"item_id": "6342039236", "options": {"switch type": "clicky",
"backlight": "white", "size": "full size"}, "available": true, "price": 244.91},
"1421289881": {"item_id": "1421289881", "options": {"switch type": "linear",
"backlight": "none", "size": "80%"}, "available": true, "price": 268.77},
"7867398203": {"item_id": "7867398203", "options": {"switch type": "linear",
"backlight": "RGB", "size": "60%"}, "available": true, "price": 232.7},
"4648814700": {"item_id": "4648814700", "options": {"switch type": "linear",
"backlight": "white", "size": "60%"}, "available": false, "price": 228.84},
"5222576926": {"item_id": "5222576926", "options": {"switch type": "linear",
"backlight": "white", "size": "full size"}, "available": false, "price": 249.95},
"4402162122": {"item_id": "4402162122", "options": {"switch type": "tactile",
"backlight": "RGB", "size": "60%"}, "available": true, "price": 233.9},
"1151293680": {"item_id": "1151293680", "options": {"switch type": "linear",
"backlight": "RGB", "size": "full size"}, "available": true, "price": 272.33},
"2299424241": {"item_id": "2299424241", "options": {"switch type": "clicky",
"backlight": "RGB", "size": "80%"}, "available": true, "price": 237.48},
"4843487907": {"item_id": "4843487907", "options": {"switch type": "clicky",
"backlight": "white", "size": "80%"}, "available": false, "price": 254.84},
"9025753381": {"item_id": "9025753381", "options": {"switch type": "clicky",
"backlight": "RGB", "size": "full size"}, "available": false, "price": 231.58},
"6439196450": {"item_id": "6439196450", "options": {"switch type": "tactile",
"backlight": "none", "size": "60%"}, "available": false, "price": 254.56},
"9991484137": {"item_id": "9991484137", "options": {"switch type": "tactile",
"backlight": "white", "size": "80%"}, "available": true, "price": 240.97},
"9665000388": {"item_id": "9665000388", "options": {"switch type": "clicky",
"backlight": "none", "size": "80%"}, "available": true, "price": 269.46},
"9570044148": {"item_id": "9570044148", "options": {"switch type": "linear",
"backlight": "none", "size": "full size"}, "available": true, "price": 231.37},
"7658724607": {"item_id": "7658724607", "options": {"switch type": "tactile",
"backlight": "none", "size": "80%"}, "available": true, "price": 256.73}}}

```

assistant: clicky switch와 RGB backlight가 있는 full-size Mechanical Keyboard로 가능한 옵션은 다음과 같습니다:

1. ****Item ID: 7706410293****
 - 스위치 유형: Clicky
 - 백라이트: None
 - 크기: Full Size
 - 가격: \$269.16
2. ****Item ID: 6342039236****
 - 스위치 유형: Clicky

- 백라이트: White
- 크기: Full Size
- 가격: \$244.91

안타깝게도 현재 clicky switch와 RGB backlight를 모두 갖춘 full-size 모델은 재고가 없습니다. 사용 가능한 옵션 중 하나로 진행하시겠습니까, 아니면 원하시는 모델이 다시 입고될 때까지 기다리시겠습니까?

user: 이해했습니다. 음, 그렇다면 clicky switch를 우선하고 backlight가 없는 full-size 모델, 즉 Item ID: 7706410293로 하겠습니다.

assistant: 좋은 선택입니다! exchange 세부사항은 다음과 같습니다:

- **현재 Item:** Mechanical Keyboard (Item ID: 1151293680)
 - 스위치 유형: Linear
 - 백라이트: RGB
 - 크기: Full Size
 - 가격: \$272.33
- **새 Item:** Mechanical Keyboard (Item ID: 7706410293)
 - 스위치 유형: Clicky
 - 백라이트: None
 - 크기: Full Size
 - 가격: \$269.16

차액은 기존 payment method인 credit card로 refund됩니다.

이 exchange를 진행하려면 확인 부탁드립니다.

user: 네, 이 exchange면 괜찮습니다. 저에게는 backlight보다 clicky switch가 훨씬 더 중요합니다. 이 exchange로 진행해 주세요.

assistant: `Function(arguments={"order_id":"#W2378156","item_ids":["1151293680"],"new_item_ids":["7706410293"],"payment_method_id":"credit_card_9513926"}, name='exchange_delivered_order_items')`

```

tool: {"order_id": "#W2378156", "user_id": "yusuf_rossi_9620", "address":
{"address1": "763 Broadway", "address2": "Suite 135", "city": "Philadelphia",
"country": "USA", "state": "PA", "zip": "19122"}, "items": [{"name": "Headphones",
"product_id": "6992792935", "item_id": "4202497723", "price": 342.81, "options":
{"type": "over-ear", "connectivity": "wireless", "color": "blue"}}, {"name":
"Vacuum Cleaner", "product_id": "1762337868", "item_id": "4602305039", "price":
561.05, "options": {"type": "robotic", "bagged/bagless": "bagged", "features":
"cordless"}}, {"name": "Mechanical Keyboard", "product_id": "1656367028",
"item_id": "1151293680", "price": 272.33, "options": {"switch type": "linear",
"backlight": "RGB", "size": "full size"}}, {"name": "Smart Thermostat",
"product_id": "4896585277", "item_id": "4983901480", "price": 262.47, "options":
{"compatibility": "Apple HomeKit", "color": "black"}}, {"name": "Smart Watch",
"product_id": "6945232052", "item_id": "9408160950", "price": 381.26, "options":
{"color": "gold", "band material": "leather", "display": "LCD"}}], "fulfillments":
[{"tracking_id": ["843053632392"], "item_ids": ["4202497723", "4602305039",
"1151293680", "4983901480", "9408160950"]}], "status": "exchange requested",
"payment_history": [{"transaction_type": "payment", "amount": 1819.92,
"payment_method_id": "credit_card_9513926"}], "exchange_items": ["1151293680"],
"exchange_new_items": ["7706410293"], "exchange_payment_method_id":
"credit_card_9513926", "exchange_price_difference": -3.17}

```

assistant: exchange 요청이 성공적으로 처리되었습니다. 세부사항은 다음과 같습니다:

- **현재 Item:** Mechanical Keyboard (Item ID: 1151293680)
 - 스위치 유형: Linear
 - 백라이트: RGB
 - 크기: Full Size
 - 가격: \$272.33
- **새 Item:** Mechanical Keyboard (Item ID: 7706410293)
 - 스위치 유형: Clicky
 - 백라이트: None
 - 크기: Full Size
 - 가격: \$269.16
- **차액:** \$3.17 (credit card로 refund됩니다)

현재 item을 반송하는 방법이 담긴 이메일을 받게 됩니다. 반송이 처리되면 새 item이 배송될 것입니다.

오늘 더 도와드릴 일이 있을까요?

user: 네, exchange에 관해 상의하고 싶은 item이 하나 더 있습니다. 제 order의 Smart Thermostat가 현재 Apple HomeKit과 호환되는데, Google Home과 호환되는 것으로 exchange하고 싶습니다.

```

assistant: Function(arguments='{ "product_id": "4896585277" }',
name='get_product_details')

```



```
tool: {"name": "Smart Thermostat", "product_id": "4896585277", "variants":  
{"8722653925": {"item_id": "8722653925", "options": {"compatibility": "Google  
Assistant", "color": "white"}, "available": false, "price": 227.8}, "8593894906":  
{"item_id": "8593894906", "options": {"compatibility": "Amazon Alexa", "color":  
"white"}, "available": false, "price": 263.11}, "2791467853": {"item_id":  
"2791467853", "options": {"compatibility": "Google Assistant", "color": "stainless  
steel"}, "available": false, "price": 242.53}, "7747408585": {"item_id":  
"7747408585", "options": {"compatibility": "Google Assistant", "color": "black"},  
"available": true, "price": 249.01}, "4953074738": {"item_id": "4953074738",  
"options": {"compatibility": "Amazon Alexa", "color": "black"}, "available": true,  
"price": 226.02}, "4983901480": {"item_id": "4983901480", "options":  
{"compatibility": "Apple HomeKit", "color": "black"}, "available": true, "price":  
262.47}, "9480266227": {"item_id": "9480266227", "options": {"compatibility":  
"Apple HomeKit", "color": "stainless steel"}, "available": true, "price": 255.98},  
"6243148452": {"item_id": "6243148452", "options": {"compatibility": "Amazon  
Alexa", "color": "stainless steel"}, "available": true, "price": 247.0},  
"3377900078": {"item_id": "3377900078", "options": {"compatibility": "Apple  
HomeKit", "color": "white"}, "available": true, "price": 260.68}}}
```

assistant: Google Assistant와 호환되는 Smart Thermostat 옵션은 다음과 같습니다:

1. ****Item ID: 7747408585****
 - 호환성: Google Assistant
 - 색상: Black
 - 가격: \$249.01

안타깝게도 이 옵션만이 고객님의 요구사항에 맞는 현재 이용 가능한 유일한 선택지입니다. 이 exchange를 진행하시겠습니까?

user: 그 item이 제가 원하던 요구사항에 맞습니다. 네, thermostat exchange를 진행해 주세요.

assistant: exchange 세부사항은 다음과 같습니다:

- ****현재 Item:**** Smart Thermostat (Item ID: 4983901480)
 - 호환성: Apple HomeKit
 - 색상: Black
 - 가격: \$262.47
- ****새 Item:**** Smart Thermostat (Item ID: 7747408585)
 - 호환성: Google Assistant
 - 색상: Black
 - 가격: \$249.01

차액은 기존 payment method인 credit card로 refund됩니다.

이 exchange를 진행하려면 확인 부탁드립니다.

user: 네, thermostat도 exchange하고 싶습니다. 저에게는 Google Assistant 호환성이 정말 중요해서 이 선택이 맞는 것 같습니다.

```
assistant: Function(arguments='{ "order_id": "#W2378156", "item_ids": ["4983901480"], "new_item_ids": ["7747408585"], "payment_method_id": "credit_card_9513926" }',  
name='exchange_delivered_order_items')
```

tool: Error: non-delivered order cannot be exchanged

```
assistant: Function(arguments='{ "summary": "User Yusuf Rossi가 Smart Thermostat(Item ID: 4983901480)를 Google Assistant 호환 모델(Item ID: 7747408585)로 exchange하고 싶어합니다. 시스템에서 order가 배송 완료로 표시되지 않았다는 error를 반환했지만, order status는 실제로 \'delivered\'입니다. exchange를 도와주세요." }',  
name='transfer_to_human_agents')
```

tool: Transfer successful

C.2.2 Task 7: 잘못된 argument

이 예시에서 agent는 복잡한 product inventory를 reasoning해 원하는 exchange item option을 찾지 못해 실패한다.

```
{
  "instruction": "당신은 mei_kovacs_8020(28236)이며 water bottle과 desk lamp를 exchange하고 싶습니다. water bottle은 더 큰 것으로, desk lamp는 더 어두운 것으로 바꾸고 싶습니다(선호도는 AC adapter > battery > USB). agent가 확인을 요청하면 desk lamp만 exchange하세요.",
  "actions": [
    {
      "name": "exchange_delivered_order_items",
      "arguments": {
        "order_id": "#W6390527",
        "item_ids": ["8384507844"],
        "new_item_ids": ["1569765161"],
        "payment_method_id": "paypal_7644869"
      }
    }
  ]
}
```

user: 안녕하세요! 귀사 매장에서 water bottle과 desk lamp를 샀는데 둘 다 exchange하고 싶습니다. 도와주실 수 있을까요?

assistant: 물론 도와드릴 수 있습니다. 먼저 신원 확인이 필요합니다. 이메일 주소를 알려주시겠어요?

user: 네, mei_kovacs_8020@mail.com입니다.

assistant: `Function(arguments='{ "email": "mei_kovacs_8020@mail.com" }', name='find_user_id_by_email')`

tool: Error: user not found

assistant: 해당 이메일 주소로는 user를 찾지 못한 것 같습니다. 대신 이름, 성, 그리고 zip code를 알려주시겠어요?

user: 아, 혼란을 드려 죄송합니다. 제 이름은 Mei Kovacs이고 zip code는 28236입니다.

assistant: `Function(arguments='{ "first_name": "Mei", "last_name": "Kovacs", "zip": "28236" }', name='find_user_id_by_name_zip')`

tool: mei_kovacs_8020

assistant: Function(arguments='{ "user_id": "mei_kovacs_8020" }',
name='get_user_details')

tool: { "name": { "first_name": "Mei", "last_name": "Kovacs" }, "address":
{ "address1": "317 Elm Street", "address2": "Suite 461", "city": "Charlotte",
"country": "USA", "province": "NC", "zip": "28236" }, "email":
"mei.kovacs8232@example.com", "payment_methods": { "paypal_7644869": { "source":
"paypal", "id": "paypal_7644869" } }, "orders": ["#W6390527", "#W7800651",
"#W8065207"] }

assistant: 감사합니다, Mei님. 신원 확인이 완료되었습니다. 이제 water bottle과 desk
lamp가 들어 있는 order 세부사항을 확인해야 합니다. 해당 item들의 order ID를
알려주시겠어요?

user: 죄송하지만 지금은 order ID가 기억나지 않습니다. 알려드릴 수가 없네요.

assistant: Function(arguments='{ "order_id": "#W6390527" }', name='get_order_details')

tool: { "order_id": "#W6390527", "user_id": "mei_kovacs_8020", "address":
{ "address1": "317 Elm Street", "address2": "Suite 461", "city": "Charlotte",
"country": "USA", "state": "NC", "zip": "28236" }, "items": [{ "name": "Desk Lamp",
"product_id": "6817146515", "item_id": "8384507844", "price": 137.94, "options":
{ "color": "white", "brightness": "medium", "power source": "USB" } }, { "name":
"Hiking Boots", "product_id": "7363354090", "item_id": "1615379700", "price":
253.89, "options": { "size": "10", "material": "synthetic", "waterproof": "yes" } },
{ "name": "Water Bottle", "product_id": "8310926033", "item_id": "8538875209",
"price": 45.13, "options": { "capacity": "500ml", "material": "glass", "color":
"black" } }], "fulfillments": [{ "tracking_id": ["745984883162"], "item_ids":
["8384507844", "1615379700", "8538875209"] }], "status": "delivered",
"payment_history": [{ "transaction_type": "payment", "amount": 436.96,
"payment_method_id": "paypal_7644869" }] }

assistant: water bottle과 desk lamp가 모두 포함된 order를 찾았습니다. 세부사항은 다음과
같습니다:

- **Order ID:** #W6390527
- **Status:** Delivered
- **Item들:**
- Desk Lamp (Item ID: 8384507844)
- Water Bottle (Item ID: 8538875209)

exchange를 진행하려면 어떤 새 item으로 바꾸고 싶은지 알아야 합니다. 새 item들의 item
ID를 알려주세요.

user: 지금 정확한 item ID는 없습니다. 대신 제가 정확히 무엇을 찾는지 말씀드려도 될까요?

assistant: 물론입니다. exchange하려는 새 item을 설명해 주시면 제가 item ID를 찾아드리겠습니다. 원하시는 새 desk lamp와 water bottle의 조건을 알려주세요.

user: 좋습니다! 현재 water bottle은 더 큰 것으로, desk lamp는 지금보다 덜 밝은 것으로 exchange하고 싶습니다. lamp는 가능하다면 AC adapter를 battery보다, 그리고 battery를 USB보다 선호합니다.

```
assistant: Function(arguments={'product_id': '8310926033'},
name='get_product_details')
```

```
tool: {"name": "Water Bottle", "product_id": "8310926033", "variants":
{"1434748144": {"item_id": "1434748144", "options": {"capacity": "1000ml",
"material": "glass", "color": "red"}, "available": false, "price": 49.72},
"4579334072": {"item_id": "4579334072", "options": {"capacity": "750ml",
"material": "glass", "color": "black"}, "available": true, "price": 54.85},
"6469567736": {"item_id": "6469567736", "options": {"capacity": "1000ml",
"material": "glass", "color": "blue"}, "available": false, "price": 47.84},
"3453331371": {"item_id": "3453331371", "options": {"capacity": "500ml",
"material": "stainless steel", "color": "black"}, "available": true, "price":
52.79}, "2439754078": {"item_id": "2439754078", "options": {"capacity": "1000ml",
"material": "stainless steel", "color": "red"}, "available": true, "price":
49.51}, "7843064651": {"item_id": "7843064651", "options": {"capacity": "750ml",
"material": "stainless steel", "color": "blue"}, "available": true, "price":
50.14}, "7918497119": {"item_id": "7918497119", "options": {"capacity": "500ml",
"material": "glass", "color": "blue"}, "available": false, "price": 54.51},
"5758737025": {"item_id": "5758737025", "options": {"capacity": "500ml",
"material": "glass", "color": "green"}, "available": true, "price": 45.09},
"7533802601": {"item_id": "7533802601", "options": {"capacity": "500ml",
"material": "stainless steel", "color": "green"}, "available": true, "price":
48.59}, "3229676465": {"item_id": "3229676465", "options": {"capacity": "500ml",
"material": "plastic", "color": "black"}, "available": true, "price": 51.94},
"2366567022": {"item_id": "2366567022", "options": {"capacity": "1000ml",
"material": "stainless steel", "color": "blue"}, "available": false, "price":
54.04}, "6974536207": {"item_id": "6974536207", "options": {"capacity": "750ml",
"material": "plastic", "color": "blue"}, "available": true, "price": 49.3},
"6777246137": {"item_id": "6777246137", "options": {"capacity": "750ml",
"material": "stainless steel", "color": "red"}, "available": true, "price":
47.76}, "8538875209": {"item_id": "8538875209", "options": {"capacity": "500ml",
"material": "glass", "color": "black"}, "available": true, "price": 45.13},
"9127591879": {"item_id": "9127591879", "options": {"capacity": "750ml",
"material": "stainless steel", "color": "black"}, "available": false, "price":
48.47}, "7661609223": {"item_id": "7661609223", "options": {"capacity": "1000ml",
"material": "stainless steel", "color": "black"}, "available": true, "price":
46.51}, "4947921075": {"item_id": "4947921075", "options": {"capacity": "750ml",
"material": "stainless steel", "color": "green"}, "available": false, "price":
49.57}, "7199146548": {"item_id": "7199146548", "options": {"capacity": "750ml",
"material": "plastic", "color": "black"}, "available": true, "price": 48.02}}}
```

```
assistant: Function(arguments={'product_id': "6817146515"},
name='get_product_details')
```

```

tool: {"name": "Desk Lamp", "product_id": "6817146515", "variants": {"9083642334":
{"item_id": "9083642334", "options": {"color": "white", "brightness": "high",
"power source": "USB"}, "available": true, "price": 164.28}, "4385534692":
{"item_id": "4385534692", "options": {"color": "white", "brightness": "high",
"power source": "AC adapter"}, "available": false, "price": 138.07}, "7624783998":
{"item_id": "7624783998", "options": {"color": "black", "brightness": "high",
"power source": "AC adapter"}, "available": true, "price": 154.17}, "1270145486":
{"item_id": "1270145486", "options": {"color": "white", "brightness": "high",
"power source": "battery"}, "available": false, "price": 144.07}, "5320792178":
{"item_id": "5320792178", "options": {"color": "black", "brightness": "medium",
"power source": "AC adapter"}, "available": true, "price": 135.24}, "5370728469":
{"item_id": "5370728469", "options": {"color": "silver", "brightness": "medium",
"power source": "USB"}, "available": true, "price": 164.97}, "6805564527":
{"item_id": "6805564527", "options": {"color": "black", "brightness": "medium",
"power source": "USB"}, "available": true, "price": 158.41}, "1569765161":
{"item_id": "1569765161", "options": {"color": "silver", "brightness": "low",
"power source": "AC adapter"}, "available": true, "price": 143.02}, "7453605304":
{"item_id": "7453605304", "options": {"color": "silver", "brightness": "low",
"power source": "battery"}, "available": true, "price": 150.01}, "9190635437":
{"item_id": "9190635437", "options": {"color": "black", "brightness": "low",
"power source": "USB"}, "available": true, "price": 153.23}, "4447749792":
{"item_id": "4447749792", "options": {"color": "white", "brightness": "medium",
"power source": "AC adapter"}, "available": false, "price": 139.8}, "8384507844":
{"item_id": "8384507844", "options": {"color": "white", "brightness": "medium",
"power source": "USB"}, "available": false, "price": 137.94}}}

```

assistant: 고객님의 선호에 맞는 water bottle과 desk lamp 옵션은 다음과 같습니다:

Water Bottle 옵션:

1. **Item ID:** 7661609223
 - 용량: 1000ml
 - 재질: Stainless Steel
 - 색상: Black
 - 가격: \$46.51
2. **Item ID:** 2439754078
 - 용량: 1000ml
 - 재질: Stainless Steel
 - 색상: Red
 - 가격: \$49.51

Desk Lamp 옵션:

1. **Item ID:** 5320792178
 - 색상: Black
 - 밝기: Medium
 - 전원 방식: AC Adapter
 - 가격: \$135.24
2. **Item ID:** 1569765161
 - 색상: Silver
 - 밝기: Low

- 전원 방식: AC Adapter
- 가격: \$143.02

어떤 item으로 exchange할지 확인해 주시고, 차액이 발생할 경우 사용할 payment method도 알려주세요.

user: desk lamp는 item ID 5320792178인 것으로 exchange하고 싶습니다. water bottle은 조금 더 생각할 시간이 필요합니다. 우선은 desk lamp exchange만 진행해 주세요.

assistant: delivered order의 경우 agent를 통한 return 또는 exchange는 한 번만 가능하다는 점을 유의해 주세요. 지금 desk lamp exchange를 진행하면 나중에는 저를 통해 water bottle을 exchange할 수 없게 됩니다.

지금 desk lamp exchange를 진행하시겠습니까, 아니면 두 item 모두 결정한 뒤에 진행하시겠습니까?

user: 알겠습니다. 그럼 desk lamp만 exchange 진행하겠습니다. 제한 사항을 이해했고 설명해 주셔서 감사합니다.

assistant: exchange 진행을 위해 세부사항을 정리하면 다음과 같습니다:

- ****현재 Desk Lamp:****
 - Item ID: 8384507844
 - 색상: White
 - 밝기: Medium
 - 전원 방식: USB
 - 가격: \$137.94
- ****새 Desk Lamp:****
 - Item ID: 5320792178
 - 색상: Black
 - 밝기: Medium
 - 전원 방식: AC Adapter
 - 가격: \$135.24

차액은 PayPal account로 refund됩니다.

이 exchange를 진행할지 확인해 주세요 (yes/no).

user: 네, 이 exchange를 진행하고 싶습니다. 감사합니다.

assistant: `Function(arguments={"order_id":"#W6390527","item_ids":["8384507844"],"new_item_ids":["5320792178"],"payment_method_id":"paypal_7644869"}, name='exchange_delivered_order_items')`

```
tool: {"order_id": "#W6390527", "user_id": "mei_kovacs_8020", "address":  
{"address1": "317 Elm Street", "address2": "Suite 461", "city": "Charlotte",  
"country": "USA", "state": "NC", "zip": "28236"}, "items": [{"name": "Desk Lamp",  
"product_id": "6817146515", "item_id": "8384507844", "price": 137.94, "options":  
{"color": "white", "brightness": "medium", "power source": "USB"}}, {"name":  
"Hiking Boots", "product_id": "7363354090", "item_id": "1615379700", "price":  
253.89, "options": {"size": "10", "material": "synthetic", "waterproof": "yes"}},  
{"name": "Water Bottle", "product_id": "8310926033", "item_id": "8538875209",  
"price": 45.13, "options": {"capacity": "500ml", "material": "glass", "color":  
"black"}}], "fulfillments": [{"tracking_id": ["745984883162"], "item_ids":  
["8384507844", "1615379700", "8538875209"]}], "status": "exchange requested",  
"payment_history": [{"transaction_type": "payment", "amount": 436.96,  
"payment_method_id": "paypal_7644869"}], "exchange_items": ["8384507844"],  
"exchange_new_items": ["5320792178"], "exchange_payment_method_id":  
"paypal_7644869", "exchange_price_difference": -2.7}
```

assistant: desk lamp에 대한 exchange 요청이 성공적으로 처리되었습니다. 현재 desk lamp를
반송하는 방법이 담긴 이메일을 받게 됩니다. 차액 \$2.70은 PayPal account로 refund됩니다.

추가로 도움이 필요하시면 언제든지 말씀해 주세요. 좋은 하루 보내세요!

user: ###STOP###

C.2.3 Task 42: compound 요청의 부분적 해결

이 task에서는 agent가 잘못된 주소를 고치기 위해 모든 order를 확인해야 한다. 그러나 agent는 user가 언급한 대로 jigsaw order 주소만 수정한다.

```
{
  "instruction": "당신의 이름은 Mei Patel이며, 주소는 445 Maple Drive, Suite 394, Fort Worth, Texas, 76165입니다. 방금 user id mei_patel_7272를 만들고 몇 가지를 주문했는데 주소를 잘못 입력했을 수도 있다는 걸 깨달았습니다. 이를 확인하고, 필요하다면 모든 order 주소와 user 주소를 고치고 싶습니다. 그 후에는 주문한 jigsaw를 확인하고 싶고, 아직 배송되지 않았다면 아이가 너무 어려워서 가장 쉬운 jigsaw(가장 쉬운 난이도, 가장 적은 조각 수)로 바꾸고 싶습니다. 기본 payment는 PayPal을 사용합니다. 당신은 말이 짧고 가끔 기억이 좋지 않지만, 예의 바랍니다.",
  "actions": [
    {
      "name": "modify_pending_order_address",
      "arguments": {
        "order_id": "#W9583042",
        "address1": "445 Maple Drive",
        "address2": "Suite 394",
        "city": "Fort Worth",
        "state": "TX",
        "country": "USA",
        "zip": "76165"
      }
    },
    {
      "name": "modify_pending_order_address",
      "arguments": {
        "order_id": "#W4082615",
        "address1": "445 Maple Drive",
        "address2": "Suite 394",
        "city": "Fort Worth",
        "state": "TX",
        "country": "USA",
        "zip": "76165"
      }
    },
    {
      "name": "modify_user_address",
      "arguments": {
        "user_id": "mei_patel_7272",
        "address1": "445 Maple Drive",
        "address2": "Suite 394",
        "city": "Fort Worth",
        "state": "TX",
        "country": "USA",
        "zip": "76165"
      }
    },
    {
      "name": "modify_pending_order_items",
      "arguments": {
        "order_id": "#W4082615",
```

```

        "item_ids": ["9779102705"],
        "new_item_ids": ["1096508426"],
        "payment_method_id": "paypal_4768213"
    }
}
],
}

```

user: 최근 order를 하면서 주소를 잘못 입력했을 수도 있는 것 같습니다. 확인해 주실 수 있을까요?

assistant: 물론 도와드릴 수 있습니다. 먼저 신원 확인이 필요합니다. 이메일 주소를 알려주시겠어요?

user: 등록에 어떤 이메일을 썼는지 잘 기억나지 않습니다. 대신 user id로 진행할 수 있을까요? mei_patel_7272입니다.

assistant: 그래도 신원 확인은 필요합니다. 이름, 성, 그리고 zip code를 알려주시겠어요? 그러면 user id를 찾는 데 도움이 됩니다.

user: 물론입니다. 제 전체 이름은 Mei Patel이고 zip code는 76165입니다.

assistant:
Function(arguments={"first_name": "Mei", "last_name": "Patel", "zip": "76165"},
name='find_user_id_by_name_zip')

tool: mei_patel_7272

assistant: 정보 감사합니다. user id 신원 확인이 성공적으로 완료되었습니다.

어떤 order의 주소를 확인하길 원하시는지 해당 order id를 알려주시겠어요?

user: 정확한 order id가 기억나지 않는데, 제 user id로 최근 order들을 찾을 수 있을까요?

assistant: Function(arguments={"user_id": "mei_patel_7272"},
name='get_user_details')

tool: {"name": {"first_name": "Mei", "last_name": "Patel"}, "address": {"address1": "443 Maple Drive", "address2": "Suite 394", "city": "Fort Worth", "country": "USA", "province": "TX", "zip": "76165"}, "email": "mei.patel3193@example.com", "payment_methods": {"credit_card_9503061": {"source": "credit_card", "brand": "visa", "last_four": "9904", "id": "credit_card_9503061"}, "paypal_4768213": {"source": "paypal", "id": "paypal_4768213"}}, "orders": [{"#W9583042", "#W4082615"]}}

assistant: 최근 order들을 찾았습니다. 세부사항은 다음과 같습니다:

1. Order ID: #W9583042
2. Order ID: #W4082615

어느 order의 주소를 확인하면 될지 알려주세요.

user: 정확히 어느 것인지는 기억나지 않지만, 최근에 주문한 jigsaw입니다. 그 order를 확인해 볼 수 있을까요?

assistant: `Function(arguments='{ "order_id": "#W9583042"}', name='get_order_details')`

```
tool: {"order_id": "#W9583042", "user_id": "mei_patel_7272", "address":
{"address1": "443 Maple Drive", "address2": "Suite 394", "city": "Fort Worth",
"country": "USA", "state": "TX", "zip": "76165"}, "items": [{"name": "Perfume",
"product_id": "6858788497", "item_id": "5421902839", "price": 328.25, "options":
{"scent family": "oriental", "size": "100ml", "gender": "men"}}, {"name":
"Dumbbell Set", "product_id": "7233192239", "item_id": "6245231688", "price":
522.03, "options": {"weight range": "30-50 lbs", "material": "iron", "set type":
"adjustable"}}, {"name": "Luggage Set", "product_id": "5426915165", "item_id":
"7160999700", "price": 499.29, "options": {"piece count": "2-piece", "color":
"red", "material": "softshell"}}, {"name": "Garden Hose", "product_id":
"6679515468", "item_id": "3230708338", "price": 99.51, "options": {"length":
"25ft", "material": "latex", "color": "green"}}], "fulfillments": [], "status":
"pending", "payment_history": [{"transaction_type": "payment", "amount": 1449.08,
"payment_method_id": "paypal_4768213"}]}
```

assistant: `Function(arguments='{ "order_id": "#W4082615"}', name='get_order_details')`

```
tool: {"order_id": "#W4082615", "user_id": "mei_patel_7272", "address":
{"address1": "443 Maple Drive", "address2": "Suite 394", "city": "Fort Worth",
"country": "USA", "state": "TX", "zip": "76165"}, "items": [{"name": "Jigsaw
Puzzle", "product_id": "1808611083", "item_id": "9779102705", "price": 54.11,
"options": {"pieces": "1000", "theme": "art", "difficulty level":
"intermediate"}}, {"name": "Backpack", "product_id": "2524789262", "item_id":
"5917587651", "price": 212.79, "options": {"color": "grey", "size": "medium",
"material": "polyester", "compartment": "laptop"}}, {"name": "Grill",
"product_id": "6819683148", "item_id": "3876764226", "price": 981.47, "options":
{"type": "electric", "size": "portable", "features": "side burner"}}, {"name":
"Perfume", "product_id": "6858788497", "item_id": "8316205423", "price": 288.75,
"options": {"scent family": "woody", "size": "30ml", "gender": "women"}}, {"name":
"Smart Watch", "product_id": "6945232052", "item_id": "2540052208", "price":
346.42, "options": {"color": "gold", "band material": "silicone", "display":
"LCD"}}, {"fulfillments": [], "status": "pending", "payment_history":
[{"transaction_type": "payment", "amount": 1883.54, "payment_method_id":
"paypal_4768213"}]}
```

assistant: jigsaw puzzle이 들어 있는 order는 Order ID: #W4082615입니다. 이 order의 현재 배송 주소는 다음과 같습니다:

- **주소 1:** 443 Maple Drive
- **주소 2:** Suite 394
- **도시:** Fort Worth
- **주:** TX
- **국가:** USA
- **우편번호:** 76165

의도하신 주소가 이것이 맞나요? 아니라면 올바른 주소를 알려주시면 업데이트해 드리겠습니다.

user: 아닙니다. 제 올바른 주소는 445 Maple Drive, Suite 394, Fort Worth, Texas, 76165입니다. 제 user 주소와 order 주소를 이것으로 수정해 주시겠어요?

assistant: 기본 user 주소와 pending order #W4082615의 배송 주소를 모두 다음의 올바른 주소로 업데이트할 수 있습니다:

- **주소 1:** 445 Maple Drive
- **주소 2:** Suite 394
- **도시:** Fort Worth
- **주:** TX
- **국가:** USA
- **우편번호:** 76165

이 주소가 맞는지 확인해 주시면 업데이트를 진행하겠습니다.

user: 네, 맞습니다. 업데이트 진행해 주세요.

```
assistant: Function(arguments='{ "user_id": "mei_patel_7272", "address1": "445 Maple Drive", "address2": "Suite 394", "city": "Fort Worth", "state": "TX", "country": "USA", "zip": "76165"}', name='modify_user_address')
```

```
tool: {"name": {"first_name": "Mei", "last_name": "Patel"}, "address": {"address1": "445 Maple Drive", "address2": "Suite 394", "city": "Fort Worth", "state": "TX", "country": "USA", "zip": "76165"}, "email": "mei.patel3193@example.com", "payment_methods": {"credit_card_9503061": {"source": "credit_card", "brand": "visa", "last_four": "9904", "id": "credit_card_9503061"}, "paypal_4768213": {"source": "paypal", "id": "paypal_4768213"}}, "orders": [{"#W9583042", "#W4082615"]}}
```

```
assistant: Function(arguments='{ "order_id": "#W4082615", "address1": "445 Maple Drive", "address2": "Suite 394", "city": "Fort Worth", "state": "TX", "country": "USA", "zip": "76165"}', name='modify_pending_order_address')
```

```
tool: {"order_id": "#W4082615", "user_id": "mei_patel_7272", "address":
{"address1": "445 Maple Drive", "address2": "Suite 394", "city": "Fort Worth",
"state": "TX", "country": "USA", "zip": "76165"}, "items": [{"name": "Jigsaw
Puzzle", "product_id": "1808611083", "item_id": "9779102705", "price": 54.11,
"options": {"pieces": "1000", "theme": "art", "difficulty level":
"intermediate"}}, {"name": "Backpack", "product_id": "2524789262", "item_id":
"5917587651", "price": 212.79, "options": {"color": "grey", "size": "medium",
"material": "polyester", "compartment": "laptop"}}, {"name": "Grill",
"product_id": "6819683148", "item_id": "3876764226", "price": 981.47, "options":
{"type": "electric", "size": "portable", "features": "side burner"}}, {"name":
"Perfume", "product_id": "6858788497", "item_id": "8316205423", "price": 288.75,
"options": {"scent family": "woody", "size": "30ml", "gender": "women"}}, {"name":
"Smart Watch", "product_id": "6945232052", "item_id": "2540052208", "price":
346.42, "options": {"color": "gold", "band material": "silicone", "display":
"LCD"}}], "fulfillments": [], "status": "pending", "payment_history":
[{"transaction_type": "payment", "amount": 1883.54, "payment_method_id":
"paypal_4768213"}]}
```

assistant: 기본 user 주소와 pending order #W4082615의 배송 주소가 모두 다음과 같이 성공적으로 업데이트되었습니다:

- **주소 1:** 445 Maple Drive
- **주소 2:** Suite 394
- **도시:** Fort Worth
- **주:** TX
- **국가:** USA
- **우편번호:** 76165

추가로 도와드릴 일이 있을까요?

user: 네, 사실 한 가지 더 있습니다. 제가 주문한 jigsaw를 확인해서 아직 배송되었는지 알려주실 수 있을까요?

assistant: #W4082615의 order status는 현재 "pending"이며, 이는 아직 처리되거나 배송되지 않았다는 뜻입니다.

더 알고 싶은 점이나 도움이 필요한 일이 있을까요?

user: 네, 주문한 jigsaw를 바꾸는 것을 생각 중입니다. 어린 아이에게 맞을 만한, 조각 수가 가장 적은 가장 쉬운 jigsaw가 무엇인지 알려주실 수 있을까요?

```
assistant: Function(arguments='{ }', name='list_all_product_types')
```

```
tool: {"Action Camera": "3377618313", "Air Purifier": "3821016478", "Backpack":  
"2524789262", "Bicycle": "9783735446", "Bluetooth Speaker": "4768869376",  
"Bookshelf": "8600330539", "Coffee Maker": "7996920482", "Cycling Helmet":  
"7765186836", "Desk Lamp": "6817146515", "Digital Camera": "8940227892", "Dumbbell  
Set": "7233192239", "E-Reader": "3801771308", "Electric Kettle": "1075968781",  
"Electric Toothbrush": "7352963235", "Espresso Machine": "4354588079", "Fleece  
Jacket": "8560156827", "Gaming Mouse": "5713490933", "Garden Hose": "6679515468",  
"Grill": "6819683148", "Headphones": "6992792935", "Hiking Boots": "7363354090",  
"Indoor Security Camera": "2985987096", "Jigsaw Puzzle": "1808611083", "LED Light  
Bulb": "2696197613", "Laptop": "4760268021", "Luggage Set": "5426915165", "Makeup  
Kit": "5149340237", "Mechanical Keyboard": "1656367028", "Notebook": "2892623495",  
"Office Chair": "4794339885", "Patio Umbrella": "9743693396", "Perfume":  
"6858788497", "Pet Bed": "2747247837", "Portable Charger": "6942297802", "Running  
Shoes": "6938111410", "Skateboard": "1968349452", "Smart Thermostat":  
"4896585277", "Smart Watch": "6945232052", "Smartphone": "1801728040", "Sneakers":  
"7471004230", "Sunglasses": "7314138884", "T-Shirt": "9523456873", "Tablet":  
"8024098596", "Tea Kettle": "9832717871", "Vacuum Cleaner": "1762337868", "Wall  
Clock": "2344688344", "Water Bottle": "8310926033", "Wireless Earbuds":  
"9924732112", "Wristwatch": "6066914160", "Yoga Mat": "4635925001"}
```

```
assistant: Function(arguments={'product_id':"1808611083"},  
name='get_product_details')
```

```

tool: {"name": "Jigsaw Puzzle", "product_id": "1808611083", "variants":
{"3614853563": {"item_id": "3614853563", "options": {"pieces": "2000", "theme":
"art", "difficulty level": "intermediate"}, "available": false, "price": 46.99},
"4772738468": {"item_id": "4772738468", "options": {"pieces": "1000", "theme":
"animals", "difficulty level": "beginner"}, "available": false, "price": 53.91},
"4068787148": {"item_id": "4068787148", "options": {"pieces": "500", "theme":
"art", "difficulty level": "intermediate"}, "available": true, "price": 52.01},
"3112842858": {"item_id": "3112842858", "options": {"pieces": "1000", "theme":
"fantasy", "difficulty level": "intermediate"}, "available": true, "price": 49.1},
"7869640094": {"item_id": "7869640094", "options": {"pieces": "2000", "theme":
"animals", "difficulty level": "expert"}, "available": false, "price": 47.59},
"1096508426": {"item_id": "1096508426", "options": {"pieces": "500", "theme":
"art", "difficulty level": "beginner"}, "available": true, "price": 46.13},
"9237024510": {"item_id": "9237024510", "options": {"pieces": "500", "theme":
"animals", "difficulty level": "expert"}, "available": true, "price": 53.53},
"5546244844": {"item_id": "5546244844", "options": {"pieces": "1500", "theme":
"art", "difficulty level": "intermediate"}, "available": true, "price": 51.59},
"1008948180": {"item_id": "1008948180", "options": {"pieces": "1000", "theme":
"art", "difficulty level": "beginner"}, "available": false, "price": 54.34},
"6245746168": {"item_id": "6245746168", "options": {"pieces": "1500", "theme":
"animals", "difficulty level": "intermediate"}, "available": true, "price": 46.0},
"7127170374": {"item_id": "7127170374", "options": {"pieces": "2000", "theme":
"fantasy", "difficulty level": "beginner"}, "available": false, "price": 52.03},
"9370300555": {"item_id": "9370300555", "options": {"pieces": "1000", "theme":
"art", "difficulty level": "expert"}, "available": false, "price": 45.9},
"9665100170": {"item_id": "9665100170", "options": {"pieces": "1500", "theme":
"animals", "difficulty level": "beginner"}, "available": true, "price": 45.39},
"4572024853": {"item_id": "4572024853", "options": {"pieces": "1000", "theme":
"animals", "difficulty level": "expert"}, "available": true, "price": 53.72},
"5645314103": {"item_id": "5645314103", "options": {"pieces": "2000", "theme":
"animals", "difficulty level": "intermediate"}, "available": true, "price":
46.19}, "9779102705": {"item_id": "9779102705", "options": {"pieces": "1000",
"theme": "art", "difficulty level": "intermediate"}, "available": false, "price":
54.11}, "9030221155": {"item_id": "9030221155", "options": {"pieces": "2000",
"theme": "art", "difficulty level": "beginner"}, "available": false, "price":
51.98}, "5172162216": {"item_id": "5172162216", "options": {"pieces": "2000",
"theme": "landscape", "difficulty level": "intermediate"}, "available": false,
"price": 48.51}}}

```

assistant: 어린 아이에게 맞는 가장 쉬운 Jigsaw Puzzle은 조각 수가 가장 적은 것입니다.
500 pieces 옵션은 다음과 같습니다:

1. **Item ID: 1096508426**
 - **테마:** Art
 - **난이도:** Beginner
 - **가격:** \$46.13
2. **Item ID: 9237024510**
 - **테마:** Animals
 - **난이도:** Expert
 - **가격:** \$53.53

현재 Jigsaw Puzzle을 이들 중 하나로 exchange하시겠습니까? 그렇다면 어떤 것을 원하는지 확인해 주시고, 차액 처리를 위한 payment method도 알려주세요.

user: Art 테마인 Item ID: 1096508426로 할 수 있을까요? 저는 보통 구매할 때 PayPal을 사용합니다.

assistant: exchange를 진행하기 위해 현재 Jigsaw Puzzle(Item ID: 9779102705)을 새 item(Item ID: 1096508426)으로 변경하고, 차액이 있으면 PayPal account를 사용하겠습니다.

exchange 세부사항은 다음과 같습니다:

- ****현재 Item:**** Jigsaw Puzzle (1000 pieces, Art theme, Intermediate difficulty)
- ****새 Item:**** Jigsaw Puzzle (500 pieces, Art theme, Beginner difficulty)
- ****Payment Method:**** PayPal (ID: paypal_4768213)

이 내용이 맞는지 확인해 주시면 exchange를 진행하겠습니다.

user: 네, 모두 맞아 보입니다. exchange를 진행해 주세요.

assistant: `Function(arguments='{ "order_id": "#W4082615", "item_ids": ["9779102705"], "new_item_ids": ["1096508426"], "payment_method_id": "paypal_4768213" }, name="modify_pending_order_items")`

```
tool: { "order_id": "#W4082615", "user_id": "mei_patel_7272", "address":
{"address1": "445 Maple Drive", "address2": "Suite 394", "city": "Fort Worth",
"state": "TX", "country": "USA", "zip": "76165"}, "items": [{"name": "Jigsaw
Puzzle", "product_id": "1808611083", "item_id": "1096508426", "price": 46.13,
"options": {"pieces": "500", "theme": "art", "difficulty level": "beginner"}},
{"name": "Backpack", "product_id": "2524789262", "item_id": "5917587651", "price":
212.79, "options": {"color": "grey", "size": "medium", "material": "polyester",
"compartment": "laptop"}}, {"name": "Grill", "product_id": "6819683148",
"item_id": "3876764226", "price": 981.47, "options": {"type": "electric", "size":
"portable", "features": "side burner"}}, {"name": "Perfume", "product_id":
"6858788497", "item_id": "8316205423", "price": 288.75, "options": {"scent
family": "woody", "size": "30ml", "gender": "women"}}, {"name": "Smart Watch",
"product_id": "6945232052", "item_id": "2540052208", "price": 346.42, "options":
{"color": "gold", "band material": "silicone", "display": "LCD"}}],
"fulfillments": [], "status": "pending (item modified)", "payment_history":
[{"transaction_type": "payment", "amount": 1883.54, "payment_method_id":
"paypal_4768213"}, {"transaction_type": "refund", "amount": 7.979999999999997,
"payment_method_id": "paypal_4768213"}]}
```

assistant: exchange가 성공적으로 완료되었습니다. 이제 order #W4082615에는 다음의 새 Jigsaw Puzzle이 포함됩니다:

- ****새 Item:**** Jigsaw Puzzle (500 pieces, Art theme, Beginner difficulty)

차액은 PayPal account로 처리되었으며, \$7.98의 refund가 진행되었습니다.

추가로 도와드릴 일이 있을까요?

user: 아니요, 그게 전부입니다. 도와주셔서 감사합니다.

assistant: 천만에요! 앞으로 더 궁금한 점이 있거나 도움이 필요하시면 언제든지 문의해주세요. 좋은 하루 보내세요!

user: ###STOP###

D Airline 예시

D.1 Data 예시

users.json과 flights.json에서 각각 가져온 예시는 다음과 같다. 모든 data는 code로 생성되며, code 대부분은 gpt-4가 생성했고 gpt-4 prompt는 저자들이 작성했다.

```
{
  "name": { "first_name": "Mia", "last_name": "Li" },
  "address": {
    "address1": "975 Sunset Drive",
    "address2": "Suite 217",
    "city": "Austin",
    "country": "USA",
    "province": "TX",
    "zip": "78750"
  },
  "email": "mia.li3818@example.com",
  "dob": "1990-04-05",
  "payment_methods": {
    "credit_card_4421486": {
      "source": "credit_card",
      "brand": "visa",
      "last_four": "7447",
      "id": "credit_card_4421486"
    },
    "certificate_4856383": {
      "source": "certificate",
      "amount": 100,
      "id": "certificate_4856383"
    },
    "certificate_7504069": {
      "source": "certificate",
      "amount": 250,
      "id": "certificate_7504069"
    },
    "credit_card_1955700": {
      "source": "credit_card",
      "brand": "visa",
      "last_four": "1907",
      "id": "credit_card_1955700"
    }
  },
  "saved_passengers": [{ "first_name": "Amelia", "last_name": "Ahmed", "dob": "1957-03-21" }],
  "membership": "gold",
  "reservations": ["N06J03", "AIXC49", "HKEG34"]
}
```

Listing 4: τ -airline의 users database entry 예시.

```

{
  "flight_number": "HAT001",
  "origin": "PHL",
  "destination": "LGA",
  "scheduled_departure_time_est": "06:00:00",
  "scheduled_arrival_time_est": "07:00:00",
  "dates": {
    ...,
    "2024-05-12": {
      "status": "landed",
      "actual_departure_time_est": "2024-05-12T05:44:00",
      "actual_arrival_time_est": "2024-05-12T06:39:00"
    },
    "2024-05-13": {
      "status": "landed",
      "actual_departure_time_est": "2024-05-13T05:30:00",
      "actual_arrival_time_est": "2024-05-13T06:32:00"
    },
    "2024-05-14": { "status": "cancelled" },
    "2024-05-15": {
      "status": "landed",
      "actual_departure_time_est": "2024-05-15T06:04:00",
      "actual_arrival_time_est": "2024-05-15T07:30:00"
    },
    "2024-05-16": {
      "status": "available",
      "available_seats": { "basic_economy": 16, "economy": 10, "business": 13
↪ },
      "prices": { "basic_economy": 87, "economy": 122, "business": 471 }
    },
    "2024-05-17": {
      "status": "available",
      "available_seats": { "basic_economy": 16, "economy": 13, "business": 9
↪ },
      "prices": { "basic_economy": 76, "economy": 189, "business": 498 }
    },
    "2024-05-18": {
      "status": "available",
      "available_seats": { "basic_economy": 2, "economy": 17, "business": 20
↪ },
      "prices": { "basic_economy": 91, "economy": 186, "business": 321 }
    },
    ...
  }
}

```

Listing 5: τ -airline의 flights database entry 예시.

```

{
  "reservation_id": "4WQ150",
  "user_id": "chen_jackson_3290",
  "origin": "DFW",
  "destination": "LAX",
  "flight_type": "round_trip",
  "cabin": "business",
  "flights": [
    {
      "origin": "DFW",
      "destination": "LAX",
      "flight_number": "HAT170",
      "date": "2024-05-22",
      "price": 883
    },
    {
      "origin": "LAX",
      "destination": "DFW",
      "flight_number": "HAT022",
      "date": "2024-05-26",
      "price": 779
    }
  ],
  "passengers": [
    { "first_name": "Chen", "last_name": "Jackson", "dob": "1956-07-07" },
    { "first_name": "Raj", "last_name": "Smith", "dob": "1967-04-01" },
    { "first_name": "Fatima", "last_name": "Martin", "dob": "1970-01-20" }
  ],
  "payment_history": [{ "payment_id": "gift_card_3576581", "amount": 4986 }],
  "created_at": "2024-05-02T03:10:19",
  "total_baggages": 5,
  "nonfree_baggages": 0,
  "insurance": "no"
}

```

Listing 6: τ -airline의 reservations database entry 예시.

D.2 Task 0: 성공한 trajectory

다음은 성공한 gpt-4o agent trajectory로, agent가 대화 전반에 걸쳐 동적인 user intent와 복잡한 flight 정보를 처리하는 예시다.

```
{
  "instruction": "당신은 mia_li_3668입니다. 5월 20일에 New York에서 Seattle로 가는 항공편을 원합니다(one way). EST 오전 11시 이전 항공편은 원하지 않습니다. economy로 비행하고 싶습니다. 직항을 선호하지만 1회 경유도 괜찮습니다. 여러 option이 있으면 가장 저렴한 것을 선호합니다. 수하물은 3개입니다. 보험은 원하지 않습니다. 결제에는 certificate 두 장을 사용하고 싶습니다. certificate를 하나만 사용할 수 있다면 더 큰 금액의 certificate를 선호하고, 나머지는 7447 카드로 지불합니다. 당신은 agent의 질문에 반응적으로 답하며, 묻지 않은 내용은 말하지 않습니다. 생년월일은 user profile에 있으므로 굳이 직접 제공하고 싶어 하지 않습니다.",
  "actions": [
    {
      "name": "book_reservation",
      "arguments": {
        "user_id": "mia_li_3668",
        "origin": "JFK",
        "destination": "SEA",
        "flight_type": "one_way",
        "cabin": "economy",
        "flights": [
          {
            "flight_number": "HAT136",
            "date": "2024-05-20",
          },
          {
            "flight_number": "HAT039",
            "date": "2024-05-20",
          },
        ],
        "passengers": [{"first_name": "Mia", "last_name": "Li", "dob": "1990-04-05"}],
        "payment_methods": [
          {"payment_id": "certificate_7504069", "amount": 250},
          {"payment_id": "credit_card_4421486", "amount": 5},
        ],
        "total_baggages": 3,
        "nonfree_baggages": 0,
        "insurance": "no",
      },
    },
  ],
}
```

user: 안녕하세요! 여행을 계획 중인데 New York에서 Seattle로 가는 flight를 booking해야 합니다. 도와주실 수 있을까요?

assistant: 물론입니다. 기꺼이 도와드리겠습니다. 먼저 user ID가 필요합니다. 알려주시겠어요?

user: 네, 제 user ID는 mia_li_3668입니다.

assistant: 좋습니다, 감사합니다! 다음으로 이번 여행에 대해 아래 정보를 알려주시겠어요?

1. Trip type: one way인가요, round trip인가요?
2. 출발 날짜는 언제인가요? (round trip이라면 돌아오는 날짜도요)
3. Cabin class는 basic economy, economy, business 중 무엇인가요?
4. 승객 수와 승객 정보(first name, last name, date of birth)는 어떻게 되나요?
5. 이번 여행에 travel insurance를 추가하시겠습니까? (승객 1인당 30달러)

이 정보를 받으면 이용 가능한 flight를 검색하겠습니다.

- user:**
1. one-way 여행입니다.
 2. 5월 20일에 출발하고 싶습니다.
 3. economy class로 비행하겠습니다.
 4. 저 혼자이고, 세부정보는 제 profile에 있을 겁니다.
 5. 아니요, travel insurance는 원하지 않습니다.

assistant: `Function(arguments='{ "user_id": "mia_li_3668" }', name='get_user_details')`

tool: `{ "name": { "first_name": "Mia", "last_name": "Li" }, "address": { "address1": "975 Sunset Drive", "address2": "Suite 217", "city": "Austin", "country": "USA", "province": "TX", "zip": "78750" }, "email": "mia.li3818@example.com", "dob": "1990-04-05", "payment_methods": { "credit_card_4421486": { "source": "credit_card", "brand": "visa", "last_four": "7447", "id": "credit_card_4421486" }, "certificate_4856383": { "source": "certificate", "amount": 100, "id": "certificate_4856383" }, "certificate_7504069": { "source": "certificate", "amount": 250, "id": "certificate_7504069" }, "credit_card_1955700": { "source": "credit_card", "brand": "visa", "last_four": "1907", "id": "credit_card_1955700" } }, "saved_passengers": [{ "first_name": "Amelia", "last_name": "Ahmed", "dob": "1957-03-21" }, { "membership": "gold", "reservations": ["N06J03", "AIXC49", "HKEG34"] }] }`

assistant:
`Function(arguments='{ "origin": "JFK", "destination": "SEA", "date": "2024-05-20" }', name='search_direct_flight')`

tool: `[{ "flight_number": "HAT069", "origin": "JFK", "destination": "SEA", "scheduled_departure_time_est": "06:00:00", "scheduled_arrival_time_est": "12:00:00", "status": "available", "available_seats": { "basic_economy": 17, "economy": 12, "business": 3 }, "prices": { "basic_economy": 51, "economy": 121, "business": 239 } }, { "flight_number": "HAT083", "origin": "JFK", "destination": "SEA", "scheduled_departure_time_est": "01:00:00", "scheduled_arrival_time_est": "07:00:00", "status": "available", "available_seats": { "basic_economy": 16, "economy": 7, "business": 3 }, "prices": { "basic_economy": 87, "economy": 100, "business": 276 } }]`

assistant: 2024년 5월 20일 New York (JFK)에서 Seattle (SEA)로 가는 이용 가능한 직항 flight는 다음과 같습니다:

1. ****Flight HAT069****
 - 출발: 06:00 AM EST
 - 도착: 12:00 PM EST
 - 이용 가능 좌석: Economy 12석
 - 가격: \$121
2. ****Flight HAT083****
 - 출발: 01:00 AM EST
 - 도착: 07:00 AM EST
 - 이용 가능 좌석: Economy 7석
 - 가격: \$100

어느 flight를 booking할지 확인해 주세요.

user: 두 flight 모두 저에게는 너무 이른 시간입니다. 가능하다면 EST 오전 11시 이전에 출발하지 않는 flight를 원합니다.

assistant:

```
Function(arguments='{"origin":"JFK","destination":"SEA","date":"2024-05-20"}',  
name='search_onestop_flight')
```

```

tool: [[{"flight_number": "HAT057", "origin": "JFK", "destination": "ATL",
"scheduled_departure_time_est": "07:00:00", "scheduled_arrival_time_est":
"09:30:00", "status": "available", "available_seats": {"basic_economy": 7,
"economy": 3, "business": 9}, "prices": {"basic_economy": 76, "economy": 141,
"business": 477}, "date": "2024-05-20"}, {"flight_number": "HAT039", "origin":
"ATL", "destination": "SEA", "scheduled_departure_time_est": "22:00:00",
"scheduled_arrival_time_est": "03:00:00+1", "status": "available",
"available_seats": {"basic_economy": 18, "economy": 10, "business": 2}, "prices":
{"basic_economy": 52, "economy": 103, "business": 283}, "date": "2024-05-20"}],
[{"flight_number": "HAT136", "origin": "JFK", "destination": "ATL",
"scheduled_departure_time_est": "19:00:00", "scheduled_arrival_time_est":
"21:30:00", "status": "available", "available_seats": {"basic_economy": 15,
"economy": 14, "business": 2}, "prices": {"basic_economy": 52, "economy": 152,
"business": 277}, "date": "2024-05-20"}, {"flight_number": "HAT039", "origin":
"ATL", "destination": "SEA", "scheduled_departure_time_est": "22:00:00",
"scheduled_arrival_time_est": "03:00:00+1", "status": "available",
"available_seats": {"basic_economy": 18, "economy": 10, "business": 2}, "prices":
{"basic_economy": 52, "economy": 103, "business": 283}, "date": "2024-05-20"}],
[{"flight_number": "HAT218", "origin": "JFK", "destination": "ATL",
"scheduled_departure_time_est": "18:00:00", "scheduled_arrival_time_est":
"20:30:00", "status": "available", "available_seats": {"basic_economy": 12,
"economy": 1, "business": 4}, "prices": {"basic_economy": 76, "economy": 158,
"business": 342}, "date": "2024-05-20"}, {"flight_number": "HAT039", "origin":
"ATL", "destination": "SEA", "scheduled_departure_time_est": "22:00:00",
"scheduled_arrival_time_est": "03:00:00+1", "status": "available",
"available_seats": {"basic_economy": 18, "economy": 10, "business": 2}, "prices":
{"basic_economy": 52, "economy": 103, "business": 283}, "date": "2024-05-20"}],
[{"flight_number": "HAT268", "origin": "JFK", "destination": "ATL",
"scheduled_departure_time_est": "07:00:00", "scheduled_arrival_time_est":
"09:30:00", "status": "available", "available_seats": {"basic_economy": 17,
"economy": 19, "business": 12}, "prices": {"basic_economy": 91, "economy": 101,
"business": 453}, "date": "2024-05-20"}, {"flight_number": "HAT039", "origin":
"ATL", "destination": "SEA", "scheduled_departure_time_est": "22:00:00",
"scheduled_arrival_time_est": "03:00:00+1", "status": "available",
"available_seats": {"basic_economy": 18, "economy": 10, "business": 2}, "prices":
{"basic_economy": 52, "economy": 103, "business": 283}, "date": "2024-05-20"}]]

```

assistant: 2024년 5월 20일 New York (JFK)에서 Seattle (SEA)로 가는 1회 경유 flight 중,
EST 오전 11시 이후에 출발하는 옵션은 다음과 같습니다:

1. **Flight 옵션 1**
 - **첫 번째 구간:**
 - Flight Number: HAT136
 - 출발: 07:00 PM EST from JFK
 - 도착: 09:30 PM EST at ATL
 - 이용 가능 좌석: Economy 14석
 - 가격: \$152
 - **두 번째 구간:**
 - Flight Number: HAT039
 - 출발: 10:00 PM EST from ATL
 - 도착: 03:00 AM EST (+1 day) at SEA
 - 이용 가능 좌석: Economy 10석
 - 가격: \$103

2. ****Flight 옵션 2****

- ****첫 번째 구간:****
 - Flight Number: HAT218
 - 출발: 06:00 PM EST from JFK
 - 도착: 08:30 PM EST at ATL
 - 이용 가능 좌석: Economy 1석
 - 가격: \$158
- ****두 번째 구간:****
 - Flight Number: HAT039
 - 출발: 10:00 PM EST from ATL
 - 도착: 03:00 AM EST (+1 day) at SEA
 - 이용 가능 좌석: Economy 10석
 - 가격: \$103

어느 flight 옵션을 booking할지 확인해 주세요.

user: 첫 번째 옵션으로 하겠습니다. JFK에서 EST 오후 07:00에 출발하는 Flight HAT136과 ATL에서 EST 오후 10:00에 출발하는 Flight HAT039입니다.

assistant: booking 세부사항은 다음과 같습니다:

- ****여정 유형:**** 편도
- ****출발지:**** New York (JFK)
- ****도착지:**** Seattle (SEA)
- ****좌석 등급:**** Economy
- ****항공편들:****
 - ****첫 번째 구간:****
 - 항공편 번호: HAT136
 - 출발: 07:00 PM EST from JFK
 - 도착: 09:30 PM EST at ATL
 - ****두 번째 구간:****
 - 항공편 번호: HAT039
 - 출발: 10:00 PM EST from ATL
 - 도착: 03:00 AM EST (+1 day) at SEA
- ****승객:****
 - Mia Li (DOB: 1990-04-05)
- ****총 가격:**** \$152 + \$103 = \$255
- ****위탁 수하물:**** 3 free checked bags (economy class의 Gold member 혜택)
- ****여행자 보험:**** No

****결제 수단:****

- 사용 가능한 payment method는 다음과 같습니다:
 - 7447로 끝나는 Credit Card
 - 1907로 끝나는 Credit Card
 - \$100 Certificate
 - \$250 Certificate

이 booking을 진행할지 확인해 주시고, 사용할 payment method를 지정해 주세요.

user: 네, booking 정보는 맞습니다. 결제에는 먼저 certificate 두 장을 사용하고 싶습니다. certificate를 하나만 사용할 수 있다면 \$250짜리를 사용하고, 나머지는 7447로 끝나는 credit card로 결제해 주세요.

assistant: 안전상의 이유로 reservation당 travel certificate는 하나만 사용할 수 있습니다. 따라서 \$250 certificate를 사용하고 나머지 금액은 7447로 끝나는 credit card로 청구하겠습니다.

결제 세부사항은 다음과 같습니다:

- **Certificate:** \$250
- **Credit Card (ending in 7447):** \$5

이 결제 방식으로 진행할지 확인해 주세요.

user: 네, 확인합니다. 이 결제 방식으로 진행해 주세요.

```
assistant: Function(arguments='{ "user_id": "mia_li_3668", "origin": "JFK", "destination": "SEA", "flight_type": "one_way", "cabin": "economy", "flights": [{ "flight_number": "HAT136", "date": "2024-05-20" }, { "flight_number": "HAT039", "date": "2024-05-20" } ], "passengers": [{ "first_name": "Mia", "last_name": "Li", "dob": "1990-04-05" } ], "payment_methods": [ { "payment_id": "certificate_7504069", "amount": 250 }, { "payment_id": "credit_card_4421486", "amount": 5 } ], "total_baggages": 3, "nonfree_baggages": 0, "insurance": "no" }', name='book_reservation')
```

```
tool: { "reservation_id": "HATHAT", "user_id": "mia_li_3668", "origin": "JFK", "destination": "SEA", "flight_type": "one_way", "cabin": "economy", "flights": [ { "flight_number": "HAT136", "date": "2024-05-20", "price": 152, "origin": "JFK", "destination": "ATL" }, { "flight_number": "HAT039", "date": "2024-05-20", "price": 103, "origin": "ATL", "destination": "SEA" } ], "passengers": [ { "first_name": "Mia", "last_name": "Li", "dob": "1990-04-05" } ], "payment_history": [ { "payment_id": "certificate_7504069", "amount": 250 }, { "payment_id": "credit_card_4421486", "amount": 5 } ], "created_at": "2024-05-15T15:00:00", "total_baggages": 3, "nonfree_baggages": 0, "insurance": "no" }
```

assistant: flight booking이 성공적으로 완료되었습니다! reservation 세부사항은 다음과 같습니다:

- **Reservation ID:** HATHAT
- **여행 유형:** 편도
- **출발지:** New York (JFK)
- **도착지:** Seattle (SEA)
- **좌석 등급:** Economy
- **항공편들:**
 - **첫 번째 구간:**
 - 항공편 번호: HAT136
 - 출발: 07:00 PM EST from JFK
 - 도착: 09:30 PM EST at ATL
 - **두 번째 구간:**
 - 항공편 번호: HAT039

- 출발: 10:00 PM EST from ATL
- 도착: 03:00 AM EST (+1 day) at SEA
- **승객:**
 - Mia Li (DOB: 1990-04-05)
- **위탁 수하물:** 3 free checked bags
- **여행자 보험:** No

****결제 세부사항:****

- ****Certificate:**** \$250
- ****Credit Card (ending in 7447):**** \$5

추가로 궁금한 점이 있거나 더 도움이 필요하시면 언제든지 말씀해 주세요. 좋은 여행 되세요!

user: ###STOP###